

BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO CUỐI KÌ

Học phần : Thực tập cơ sở

Lớp: D23CQCE06-B

Họ Tên: Đinh Tuấn Thành

Mã sv: B23DCCE090

Hướng dẫn: Kim Ngọc Bách

Mục Lục

I.ĐỀ CƯƠNG	3
1. GIỚI THIỆU DỰ ÁN	3
2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	3
3. PHÂN TÍCH YÊU CẦU DỰ ÁN	3
4. KẾ HOẠCH THỰC HIỆN DỰ ÁN.....	4
II.BÁO CÁO CUỐI KÌ.....	4
1. GIỚI THIỆU CHUNG.....	4
1.1. Đặt vấn đề	5
1.2. Ý nghĩa thực tiễn đối với người dùng đặc biệt	5
1.3. Tổng quan hệ thống FruitAI	6
1.4. Các loại trái cây được hỗ trợ nhận diện	6
2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	7
2.1. Mô hình YOLOv8 và kỹ thuật Fine-tuning	7
2.2. Thuật toán phân tích độ chín qua không gian màu HSV	10
2.3. Cơ sở dữ liệu dinh dưỡng (nutrition_database.json).....	16
2.4. Flask AI Service.....	17
2.5. Node.js Backend — Điều phối nghiệp vụ	19
2.6. Controllers — Cài đặt chi tiết logic nghiệp vụ	23
2.7. Cơ sở dữ liệu MySQL.....	32
2.8. Biến môi trường và script khởi động	34
3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	36
3.1. Kiến trúc tổng thể (Microservice 3-tier)	36
3.2. Luồng dữ liệu chính	36
3.3. Phân tích yêu cầu hệ thống	37
3.4. Cấu trúc thư mục dự án.....	37
4. KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM.....	38
4.1. Môi trường triển khai.....	38
4.2. Kết quả giao diện hệ thống	38
4.3. Bảng kết quả thử nghiệm nhận diện	41
4.4. Các trường hợp biên đã xử lý.....	41
5. KẾT LUẬN.....	42
5.1. Những kết quả đạt được	42
5.2. Hạn chế còn tồn tại	42
5.3. Hướng phát triển trong tương lai	42

5.4. Kết luận chung	43
TÀI LIỆU TRÍCH DẪN	44

I. ĐỀ CƯƠNG

1. GIỚI THIỆU DỰ ÁN

- Lý do chọn đề tài : Trong thời đại số hóa y tế, việc kiểm soát chế độ ăn uống cá nhân ngày càng quan trọng. Tuy nhiên, các ứng dụng hiện nay thường chỉ dừng lại ở việc nhận diện tên loại quả chung chung. Dự án này được phát triển nhằm giải quyết bài toán định lượng: "Nhận diện đúng giống loài + Đánh giá độ chín + Tính toán chính xác hơn 15 chỉ số dinh dưỡng dựa trên khối lượng thực tế".
- Ý nghĩa : Cung cấp một công cụ phân tích thực phẩm toàn diện, giúp người dùng hiểu rõ giá trị dinh dưỡng của từng cá thể trái cây cụ thể thay vì các con số lý thuyết trung bình.
- Tính ứng dụng :
 - + Ứng dụng di động hỗ trợ người bệnh tiểu đường và người tập Gym.
 - + Hệ thống kiểm tra chất lượng nông sản tự động cho các cửa hàng thực phẩm sạch.
 - + Trợ lý dinh dưỡng ảo tích hợp trong các thiết bị nhà bếp thông minh.
- Giá trị học thuật : Nghiên cứu ứng dụng kỹ thuật Transfer Learning (Học chuyển đổi) để tối ưu hóa mô hình nhận diện trái cây đặc hữu và kết hợp thuật toán xử lý ảnh truyền thống với hệ thống suy diễn dữ liệu định lượng cục bộ.

2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

- Nhận dạng thực thể bằng YOLOv8 : Sử dụng kiến trúc YOLOv8 (You Only Look Once v8) để thực hiện nhận diện và phân đoạn thực thể. YOLOv8 vượt trội về tốc độ xử lý thời gian thực và độ chính xác trong việc tách biệt ranh giới vật thể.
- Kỹ thuật Tinh chỉnh mô hình (Fine-tuning & Transfer Learning): Thay vì huấn luyện từ đầu, dự án sử dụng phương pháp Transfer Learning để tinh chỉnh mô hình YOLOv8 trên tập dữ liệu trái cây chuyên biệt. Kỹ thuật này giúp mô hình đạt độ chính xác cao ngay cả với tập dữ liệu nhỏ bằng cách kế thừa các đặc trưng hình ảnh từ bộ dữ liệu lớn COCO .
- Phân tích độ chín qua không gian màu HSV : Sử dụng thuật toán trích xuất biểu đồ màu (Color Histogram) trong không gian màu HSV (Hue, Saturation, Value) để xác định trạng thái chín của quả dựa trên sự thay đổi sắc thái vỏ, giúp điều chỉnh các tham số đường và calo trong tính toán.
- Cơ sở dữ liệu dinh dưỡng đa chỉ số (Self-built Database) : Xây dựng hệ thống dữ liệu cục bộ dựa trên định mức chuẩn của USDA FoodData Central. Dữ liệu bao gồm các nhóm Đa lượng (Macros) và Vi lượng (Micros) như Vitamin C, Kali, Magie, Sắt....
- Công nghệ triển khai : Ngôn ngữ Python, thư viện OpenCV (xử lý ảnh), PyTorch (huấn luyện AI), và Pandas (quản lý dữ liệu dinh dưỡng).

3. PHÂN TÍCH YÊU CẦU DỰ ÁN

Yêu cầu chức năng:

- Nhận diện : Phân biệt chính xác các giống trái cây (ví dụ: Táo đỏ vs Táo xanh).
- Tinh chỉnh (Fine-tuning) : Mô hình được huấn luyện lại để nhận diện các loại quả đặc thù theo môi trường thực tế (ánh sáng, phông nền).
- Nhập liệu : Cho phép người dùng nhập khối lượng thực tế (gram).
- Phân tích chi tiết: Xuất ra bảng thông số bao gồm :
 1. Nhóm đa lượng: Calo, Protein, Carbs, Fat, Sugar, Fiber.

2. Nhóm vi lượng: Vitamin A, C, B6, Kali, Magie, Canxi, Sắt, Kẽm.
3. Độ chín: Dự đoán % chín và trạng thái (Xanh/Chín/Quá chín).

Yêu cầu kỹ thuật :

- Hệ thống chạy offline hoàn toàn để đảm bảo tốc độ và quyền riêng tư.
- Sai số tính toán dinh dưỡng so với định mức chuẩn $< 1\%$
- Mô hình AI đạt chỉ số mAP (mean Average Precision) $> 90\%$.

4. KẾ HOẠCH THỰC HIỆN DỰ ÁN

Giai đoạn	Nội dung công việc	Công cụ thực hiện
Tuần 1	Xây dựng Database dinh dưỡng 20-30 chất/loại quả từ nguồn USDA.	Excel, JSON
Tuần 2	Thu thập và gán nhãn ảnh (Labeling) cho các trạng thái chín khác nhau.	Roboflow
Tuần 3	Tinh chỉnh mô hình (Fine-tuning) YOLOv8 trên Google Colab dùng GPU.	PyTorch, YOLOv8
Tuần 4	Lập trình module OpenCV để phân tích sắc thái màu (HSV) đánh giá độ chín.	Python, OpenCV
Tuần 5	Xây dựng thuật toán logic: Nhận diện Đánh giá chín → Tính toán theo khối lượng.	Python Backend
Tuần 6-7	Thiết kế giao diện Dashboard hiển thị thông số chi tiết và biểu đồ dinh dưỡng.	Streamlit/ Flask
Tuần 8	Kiểm thử hệ thống, đo lường sai số và hoàn thiện báo cáo thuyết trình.	

TÀI LIỆU TRÍCH DẪN

G. Jocher et al., "YOLOv8 by Ultralytics," 2023. Available: <https://github.com/ultralytics/ultralytics>.

S. J. Pan and Q. Yang, "A Survey on Transfer Learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345- 1359, 2010.

S. N. S. Al-Sarayreh et al., "Determining the ripeness of fruits using deep learning and color analysis," Journal of Food Engineering, vol. 264, 2020.

U.S. Department of Agriculture (USDA), "FoodData Central API & Database," 2023. Available: <https://fdc.nal.usda.gov/>.

II. BÁO CÁO CUỐI KÌ

1. GIỚI THIỆU CHUNG

1.1. Đặt vấn đề

Trong bối cảnh y tế số phát triển mạnh mẽ, việc kiểm soát chế độ dinh dưỡng cá nhân ngày càng đóng vai trò quan trọng trong phòng ngừa và quản lý bệnh mãn tính. Trái cây là nhóm thực phẩm thiết yếu trong bữa ăn hằng ngày của người Việt Nam, tuy nhiên thành phần dinh dưỡng của từng loại quả có sự biến động đáng kể phụ thuộc vào giống loài và trạng thái chín.

Các ứng dụng nhận diện thực phẩm hiện tại trên thị trường có nhiều hạn chế nghiêm trọng:

- Chỉ nhận diện tên loại quả chung chung, không phân biệt trạng thái chín.
- Không tích hợp dữ liệu dinh dưỡng theo từng loại quả cụ thể.
- Không có cảnh báo cho nhóm người dùng có nhu cầu đặc biệt về dinh dưỡng và sức khỏe.
- Yêu cầu kết nối internet liên tục, gây lo ngại về quyền riêng tư dữ liệu ảnh.

Đây chính là khoảng trống thực tiễn mà dự án **FruitAI** hướng đến lấp đầy.

1.2. Ý nghĩa thực tiễn đối với người dùng đặc biệt

1.2.1. Người mắc bệnh Tiểu đường (Đái tháo đường Type 1 & 2)

Người tiểu đường cần kiểm soát nghiêm ngặt lượng carbohydrate và đường đơn (fructose, glucose) nạp vào mỗi ngày. Trạng thái chín của trái cây ảnh hưởng trực tiếp đến hàm lượng đường:

Loại quả	Trạng thái	Hàm lượng đường (g/100g)
Chuối	Xanh (unripe)	~6 g
Chuối	Chín vừa (ripe)	~12 g
Chuối	Chín quá (overripe)	~16–18 g
Xoài	Xanh	~5 g
Xoài	Chín	~13.6 g

Người tiểu đường cần biết chính xác loại quả và mức độ chín để lựa chọn phù hợp với chỉ số đường huyết mục tiêu. Các loại có đường cao như **xoài chín, chuối chín quá, mít** cần hạn chế đặc biệt.

1.2.2. Người bị Dị ứng thực phẩm và Kích ứng da

Một số hợp chất tự nhiên trong trái cây có thể gây phản ứng dị ứng ngay cả với lượng nhỏ:

Hợp chất	Có trong loại quả	Triệu chứng dị ứng
Histamine	Dâu tây, chuối, cam	Nổi mề đay, ngứa, đỏ da
Latex-fruit syndrome	Bơ, chuối, dứa, đu đủ	Sưng môi, khó thở
Salicylate	Táo, cam, dứa	Viêm mũi, hen suyễn
Bromelain	Dứa (đặc biệt khi còn xanh)	Kích ứng miệng, dạ dày
Papain	Đu đủ	Dị ứng latex chéo

Người có tiền sử dị ứng cần nhận diện chính xác loại quả trước khi tiêu thụ — đây là nhu cầu thực tế mà hệ thống đáp ứng.

1.2.3. Người bệnh Suy thận mãn tính

Người suy thận cần hạn chế nghiêm ngặt lượng ****Kali (Potassium)****, vì thận suy yếu không thể đào thải Kali đủ nhanh, gây tăng kali máu — một tình trạng nguy hiểm đến tim mạch. Các loại trái cây cần đặc biệt hạn chế:

Loại quả	Kali (mg/100g)	Mức độ cần kiểm soát
----------	----------------	----------------------

Bơ	485 mg	Rất cao — hạn chế tối đa
Chuối	358 mg	Cao — hạn chế
Ổi	417 mg	Rất cao — hạn chế tối đa
Mít	448 mg	Rất cao — hạn chế tối đa
Cam	181 mg	Trung bình — kiểm soát lượng
Táo	107 mg	Thấp — tương đối an toàn

1.2.4. Người tập thể hình (Gym/Athlete)

Vận động viên cần nạp đúng loại carbohydrate vào đúng thời điểm: carbohydrate nhanh (đường đơn) để phục hồi sau tập, carbohydrate chậm (tinh bột + chất xơ) trong bữa ăn bình thường. FruitAI giúp họ tra cứu nhanh chóng protein, carbs, calo và chất xơ của từng loại quả.

1.3. Tổng quan hệ thống FruitAI

FruitAI là hệ thống web full-stack với 3 tầng dịch vụ độc lập:

- **AI Service (Python/Flask — Port 5001):** Nhận ảnh, chạy model YOLOv8, phân tích HSV, trả về kết quả JSON.
- **Backend (Node.js/Express — Port 5000):** Xác thực người dùng JWT, điều phối luồng dữ liệu, lưu trữ MySQL, phát sự kiện Socket.IO.
- **Frontend (React/Vite — Port 5173):** Giao diện người dùng, webcam realtime, biểu đồ thống kê, xuất báo cáo.

1.4. Các loại trái cây được hỗ trợ nhận diện

Hệ thống nhận diện 10 loại trái cây phổ biến:

STT	Tên (EN)	Tên (VN)	Cảnh báo sức khỏe
1	Apple	Táo	Salicylate — kích ứng hen suyễn; ít Kali — tốt cho thận
2	Avocado	Bơ	Kali cực cao (485mg) — người suy thận hạn chế; Latex-fruit
3	Banana	Chuối	Histamine & Latex-fruit; Đường cao khi chín quá
4	Dragon Fruit	Thanh Long	An toàn, ít đường — phù hợp tiêu đường và suy thận
5	Lemon	Chanh	Axit citric — kích ứng dạ dày; giàu Vitamin C
6	Mango	Xoài	Đường rất cao khi chín (13.66g) — tiểu đường hạn chế
7	Orange	Cam	Salicylate + Histamine; giàu Vitamin C (53.2mg)
8	Papaya	Đu đủ	Enzyme Papain — dị ứng latex; giàu Vitamin A (950 IU)
9	Pineapple	Dứa	Bromelain — kích ứng; Latex-fruit; Salicylate
10	Strawberry	Dâu Tây	Histamine cao nhất — dễ gây dị ứng, nổi mẩn đỏ

2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Mô hình YOLOv8 và kỹ thuật Fine-tuning

2.1.1. Kiến trúc YOLOv8

YOLOv8 (You Only Look Once version 8) là kiến trúc mạng nơ-ron tích chập (CNN) thế hệ mới nhất do Ultralytics phát hành năm 2023. Tên gọi phản ánh triết lý thiết kế cốt lõi: toàn bộ ảnh chỉ được đưa qua mạng **một lần duy nhất** để dự đoán đồng thời vị trí và nhãn phân loại của tất cả vật thể.

Các điểm nổi bật của YOLOv8 so với phiên bản trước:

- **Backbone C2f (Cross Stage Partial with two convolutions):** Kết hợp các lớp tích chập dư giúp trích xuất đặc trưng đa tầng hiệu quả hơn, đặc biệt tốt với vật thể nhỏ.
- **Anchor-Free Detection Head:** Không cần định nghĩa sẵn các anchor box cố định. Mô hình tự học trực tiếp tọa độ tâm bounding box, giảm đáng kể siêu tham số cần tinh chỉnh.
- **Decoupled Head:** Đầu ra phân loại và hồi quy tọa độ được tách biệt thành hai nhánh riêng, giúp tối ưu hóa từng nhiệm vụ một cách độc lập.

2.1.2. Transfer Learning và Fine-tuning

Transfer Learning (Học chuyển đổi) là kỹ thuật tận dụng kiến thức đã học từ một tác vụ lớn để áp dụng vào tác vụ cụ thể hơn. Thay vì huấn luyện mô hình từ đầu — điều đòi hỏi hàng trăm nghìn ảnh và nhiều ngày tính toán GPU — dự án áp dụng phương pháp Fine-tuning:

- **Mô hình nguồn:** YOLOv8 đã được huấn luyện sẵn trên tập COCO (Common Objects in Context — 330,000 ảnh, 80 lớp vật thể). Mô hình này đã học được các đặc trưng hình ảnh tổng quát như phát hiện cạnh, góc, kết cấu bề mặt — những kiến thức này hoàn toàn tái sử dụng được.
- **Tác vụ đích:** Tinh chỉnh (fine-tune) lại toàn bộ mô hình trên tập dữ liệu 10 lớp trái cây chuyên biệt, được thu thập và gán nhãn qua nền tảng Roboflow, huấn luyện trên Google Colab với GPU.
- **Kết quả:** File trọng số tốt nhất trong toàn bộ quá trình training được lưu thành file `'best.pt'`

2.1.3. Nạp model và chạy suy luận

Toàn bộ logic AI được đặt trong file `'ai_service/model_service.py'`. Đầu tiên, hệ thống định nghĩa các hằng số đường dẫn và cấu hình. Đáng chú ý là cách dùng `'Path(__file__).parent.parent'` để tính động đường dẫn tới thư mục gốc dự án — nghĩa là dù chạy từ thư mục nào, chương trình vẫn tìm đúng file `'best.pt'`:

```
python
# Tính đường dẫn động: __file__ là model_service.py trong ai_service/
# .parent → ai_service/ .parent.parent → d:\aicode\
BASE_DIR = Path(__file__).parent.parent
MODEL_PATH = BASE_DIR / "best.pt" # d:\aicode\best.pt
NUTRITION_DB_PATH = BASE_DIR / "nutrition_database.json"

# Chỉ chấp nhận detection có độ tin cậy từ 35% trở lên.
# Ngưỡng này được chọn thấp hơn mặc định (50%) để phát hiện được quả
# trong điều kiện ánh sáng kém hoặc góc chụp không thuận lợi.
CONFIDENCE_THRESHOLD = 0.35
```



```
# Danh sách 10 lớp – thứ tự phải khớp chính xác với thứ tự khi huấn luyện.
# Nếu thứ tự này sai, model sẽ nhận nhầm toàn bộ tên quả.
CLASS_NAMES = [
    'apple', 'avocado', 'banana', 'dragon fruit',
    'lemon', 'mango', 'orange', 'papaya',
    'pineapple', 'strawberry'
]
```

Để tránh nạp model nhiều lần (mỗi lần khoảng 2–5 giây), hệ thống áp dụng **Singleton Pattern** model chỉ được nạp một lần khi Flask server khởi động, sau đó dùng lại cho tất cả các request:

```
python
# Ba biến toàn cục này chỉ được khởi tạo một lần duy nhất
_model: Optional[YOLO] = None # Instance model YOLOv8
_nutrition_db: dict = {} # Dict chứa dữ liệu dinh dưỡng đã đọc từ JSON
_analyzer = RipenessAnalyzer(roi_shrink=0.15) # Bộ phân tích độ chín

def load_resources():
    global _model, _nutrition_db
    # Kiểm tra file model tồn tại trước khi nạp
    if not MODEL_PATH.exists():
        raise FileNotFoundError(f"Model not found: {MODEL_PATH}")

    print(f"[AI] Loading model from {MODEL_PATH} ...")
    _model = YOLO(str(MODEL_PATH)) # Nạp model YOLOv8 từ file .pt
    print("[AI] ✓ Model loaded.")

    # Nạp cơ sở dữ liệu dinh dưỡng (nếu tồn tại)
    if NUTRITION_DB_PATH.exists():
        with open(NUTRITION_DB_PATH, "r", encoding="utf-8") as f:
            _nutrition_db = json.load(f)
        print(f"[AI] ✓ Loaded {len(_nutrition_db)} nutrition entries.")
    else:
        print("[AI] ⚠ Nutrition DB not found.")
```

Hàm `predict_image()` được gọi mỗi khi có ảnh cần nhận diện. Kết quả model được lọc và chọn detection có confidence cao nhất:

```
python
def predict_image(image_bytes: bytes) -> dict:
    if _model is None:
        raise RuntimeError("Model not loaded. Call load_resources() first.")

    # np.frombuffer: chuyển bytes thô thành mảng số nguyên 1D kiểu uint8
    # cv2.imdecode: giải mã mảng đó thành ảnh BGR 3 kênh màu (định dạng OpenCV)
    nparr = np.frombuffer(image_bytes, np.uint8)
    image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if image is None:
```

```
# Trả về lỗi nếu bytes không phải định dạng ảnh hợp lệ
raise ValueError("Cannot decode image. Invalid format.")
```

Tiếp theo, ảnh được đưa qua model YOLOv8 với các tham số được cân chỉnh cẩn thận:

```
python
results = _model.predict(
    source=image,
    conf=CONFIDENCE_THRESHOLD, # Bỏ qua detection có confidence < 35%
    iou=0.45, # Ngưỡng IoU cho Non-Maximum Suppression (NMS):
               # hai bounding box có vùng chồng chéo > 45% → giữ cái tốt
    augment=True, # Bật Test-Time Augmentation (TTA): model chạy nhiều
                  # với ảnh được flip, scale khác nhau rồi tổng hợp kết
                  # quả
                  # → tăng độ chính xác ~3-5%, đặc biệt với ảnh
                  # nghiêng/mờ
    verbose=False # Tắt log chi tiết để không làm rối output server
)
```

Sau khi có kết quả từ model, hệ thống thu thập tất cả các detection và chọn ra quả có độ tin cậy cao nhất:

```
python
detections = []
for result in results:
    if result.boxes is None:
        continue
    for box in result.boxes:
        cid = int(box.cls[0]) # Chỉ số lớp (0=apple, 1=avocado,
        ...)
        if cid >= len(CLASS_NAMES):
            continue # Bỏ qua nếu ID vượt quá số lớp
        conf = float(box.conf[0]) # Confidence: giá trị thực từ 0.0
        coords = box.xyxy[0].tolist() # Tọa độ: [x_trái, y_trên, x_phải,
        y_dưới]
        detections.append((cid, conf, coords))

    if not detections:
        # Không tìm thấy quả nào vượt ngưỡng confidence → trả về thông báo
        return {"success": False, "message": "No fruit detected in the
        image."}

# Sắp xếp giảm dần theo confidence, lấy detection đầu tiên (cao nhất)
detections.sort(key=lambda x: x[1], reverse=True)
cid, conf, coords = detections[0]
class_name = CLASS_NAMES[cid]
```

Đây chính là điểm hạn chế kỹ thuật khi ảnh chứa nhiều quả, toàn bộ kết quả ngoài quả đầu tiên bị loại bỏ.

2.2. Thuật toán phân tích độ chín qua không gian màu HSV

2.2.1. Lý thuyết không gian màu HSV

RGB (Red-Green-Blue) là không gian màu phổ biến trong máy tính, nhưng rất khó để phân tích màu sắc vì ba kênh R, G, B đều thay đổi đồng thời khi ánh sáng thay đổi. HSV (Hue – Saturation – Value) khắc phục điều này bằng cách tách biệt thông tin màu sắc ra khỏi thông tin độ sáng:

Kênh	Ý nghĩa	Phạm vi (OpenCV)	Vai trò trong bài toán
H (Hue)	Sắc màu cơ bản	0–180	Xác định màu chủ đạo của vỏ quả
S (Saturation)	Độ bão hòa màu	0–255	Lọc điểm ảnh nhạt màu (trắng/xám/nền)
V (Value)	Độ sáng	0–255	Lọc vùng quá tối (bóng) hoặc chói sáng

2.2.2. Lớp RipenessAnalyzer và cấu trúc dữ liệu kết quả

Lớp 'RipenessAnalyzer' đóng gói toàn bộ logic phân tích độ chín. Kết quả trả về là một 'dataclass' với 4 trường rõ ràng, giúp code phía sau dễ truy xuất:

```
python
@dataclass
class RipenessResult:
    stage: str # Mã trạng thái: 'unripe' | 'ripe' | 'overripe' | 'unknown'
    label: str # Nhãn hiển thị tiếng Việt: 'Chưa chín' | 'Đã chín' | 'Quá chín'
    dominant_hue: float # Giá trị Hue chủ đạo đo được (0.0–180.0), dùng để debug
    score: float # Điểm tin cậy của kết quả phân tích (0.0–1.0)

class RipenessAnalyzer:
    def __init__(self, roi_shrink: float = 0.15):
        # roi_shrink = 0.15 nghĩa là thu nhỏ 15% mỗi cạnh của bounding box
        # để chỉ phân tích phần lõi vỏ quả, tránh lấy nhầm màu nền
        self.roi_shrink = roi_shrink
```

Phương thức chính 'analyze()' điều phối toàn bộ pipeline 4 bước:

```
python
def analyze(self, frame: np.ndarray, bbox: list, class_name: str) -> RipenessResult:
    roi = self._crop_roi(frame, bbox) # Bước 1: cắt vùng quả
    if roi is None or roi.size == 0:
        return self._unknown() # Bounding box quá nhỏ/rỗng
    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV) # Chuyển từ BGR sang HSV
    mask = self._valid_pixel_mask(hsv) # Bước 2: lọc pixel nhiễu
```

```

        if cv2.countNonZero(mask) < MIN_VALID_PIXELS:
            return self._unknown() # Quá ít pixel hợp lệ →
không phân tích
        dominant_hue = self._dominant_hue(hsv[:, :, 0], mask) # Bước 3: tính
màu chủ đạo
        return self._classify(class_name.lower(), dominant_hue) # Bước 4:
phân loại

```

2.2.3. Bước 1 — Cắt vùng quan tâm (ROI)

Bounding box từ YOLOv8 bao quanh toàn bộ quả kể cả phần nền xung quanh. Nếu phân tích toàn bộ vùng đó, màu nền sẽ làm sai lệch kết quả. Kỹ thuật thu nhỏ ROI 15% mỗi cạnh giải quyết điều này:

```

python
def _crop_roi(self, frame, bbox):
    x1, y1, x2, y2 = map(int, bbox)
    h_f, w_f = frame.shape[:2]
    # Clamp tọa độ vào trong giới hạn frame.
    # Cần thiết vì đôi khi bounding box từ model vượt ra ngoài biên ảnh.
    x1 = max(0, x1); y1 = max(0, y1)
    x2 = min(w_f, x2); y2 = min(h_f, y2)
    w = x2 - x1; h = y2 - y1

    # Bounding box quá nhỏ (dưới 10x10 px) → không đủ thông tin màu sắc
    if w < 10 or h < 10:
        return None

    # Thu nhỏ 15% mỗi cạnh: ví dụ bounding box 100x80 px
    # → cắt vào 15px mỗi cạnh trái/phải, 12px mỗi cạnh trên/dưới
    # → chỉ còn lại vùng 70x56 px là lõi quả thực sự
    px = int(w * self.roi_shrink)
    py = int(h * self.roi_shrink)
    ix1, iy1 = x1 + px, y1 + py
    ix2, iy2 = x2 - px, y2 - py

    # Nếu sau khi thu nhỏ vùng bị đảo ngược (ảnh quá nhỏ) → dùng nguyên
bbox
    if ix2 <= ix1 or iy2 <= iy1:
        return frame[y1:y2, x1:x2]

    return frame[iy1:iy2, ix1:ix2]

```

2.2.4 Bước 2 — Lọc pixel nhiễu bằng mask

Không phải mọi pixel trong vùng ROI đều phản ánh màu vỏ quả. Nền, bóng đổ, vùng phản chiếu ánh sáng đều gây nhiễu. Bốn ngưỡng sau được thiết lập để lọc chỉ giữ lại các pixel màu thực sự có ý nghĩa:

```
python
```

```

    SAT_MIN          = 40    # Pixel có Saturation < 40: màu quá nhạt (trắng,
xám) → nền/ánh sáng
    VAL_MIN          = 40    # Pixel có Value < 40: quá tối → bóng đổ, vùng tối
    VAL_MAX          = 235   # Pixel có Value > 235: quá sáng → vùng bị cháy
sáng (highlight)
    MIN_VALID_PIXELS = 50    # Cần ít nhất 50 pixel hợp lệ mới phân tích; nếu
ít hơn → "Không rõ"

    def _valid_pixel_mask(self, hsv):
        # Lấy riêng kênh S và V từ ảnh HSV (kích thước HxW)
        s = hsv[:, :, 1]
        v = hsv[:, :, 2]
        # Tạo mặt nạ boolean: pixel nào ĐỒNG THỜI thỏa cả 3 điều kiện mới được
giữ lại
        mask = (s >= SAT_MIN) & (v >= VAL_MIN) & (v <= VAL_MAX)
        # Chuyển sang uint8 (0 hoặc 255) để dùng được với cv2.calcHist()
        return mask.astype(np.uint8) * 255

```

2.2.5 Bước 3 — Tính màu chủ đạo qua Color Histogram

Sau khi lọc pixel nhiễu, bước này tìm ra màu Hue xuất hiện nhiều nhất trong vùng vỏ quả. Kỹ thuật dùng là biểu đồ tần suất màu (Color Histogram):

```

python
    def _dominant_hue(self, hue_ch, mask):
        # Chia toàn bộ dải màu [0, 180] thành 36 "thùng" (bins), mỗi thùng
rộng 5 độ.
        # cv2.calcHist đếm số pixel trong mỗi thùng, chỉ tính pixel có
mask=255.
        hist = cv2.calcHist([hue_ch], [0], mask, [36], [0, 180])

        # Tìm thùng có số pixel nhiều nhất — đó là màu chủ đạo
        peak_bin = int(np.argmax(hist.flatten()))

        # Lấy điểm giữa của thùng đó làm đại diện.
        # Ví dụ: peak_bin=4 → khoảng [20°, 25°] → trung điểm = 22.5° (màu vàng
cam → chuối chín)
        # Ví dụ: peak_bin=2 → khoảng [10°, 15°] → trung điểm = 12.5° (cam
nhạt)
        return peak_bin * 5.0 + 2.5

```

2.2.6 Bước 4 — Phân loại độ chín theo luật heuristic

Giá trị Hue chủ đạo vừa tính được so sánh với bảng quy tắc 'RIPENESS_RULES' để xác định trạng thái chín. Thứ tự kiểm tra là: overripe trước, ripe sau, mặc định là unripe - nghĩa là hệ thống ưu tiên cảnh báo trạng thái xấu nhất:

```

python
    def _classify(self, class_name, hue):
        rules = RIPENESS_RULES.get(class_name)
        if rules is None:

```

```

        return self._unknown() # Loại quả chưa có quy tắc → không thể
phân tích

# Hàm kiểm tra hue có nằm trong khoảng [lo, hi] không
def in_range(h, lo, hi): return lo <= h <= hi

# --- Kiểm tra "Quá chín" trước ---
# Lý do: cần ưu tiên cảnh báo người dùng nếu quả đã hỏng
o = rules.get("overripe", {})
o_h = o.get("hue", (999, 999)) # Khoảng Hue chính cho overripe
o_w = rules.get("overripe_wrap") # Khoảng Hue phụ (màu đỏ bao
quanh 0°/180°)
if in_range(hue, *o_h) or (o_w and in_range(hue, *o_w)):
    return RipenessResult(
        o.get("stage", "overripe"), # Mã trạng thái
        o.get("label", "Quá chín"), # Nhãn tiếng Việt
        hue, 0.85 # Điểm tin cậy 85%
    )

# --- Kiểm tra "Đã chín" ---
r = rules.get("ripe", {})
r_h = r.get("hue", (999, 999))
r_w = rules.get("ripe_wrap") # Khoảng Hue phụ cho màu đỏ
chín
if in_range(hue, *r_h) or (r_w and in_range(hue, *r_w)):
    return RipenessResult(
        r.get("stage", "ripe"),
        r.get("label", "Đã chín"),
        hue, 0.90 # Điểm tin cậy cao nhất: 90%
    )

# --- Mặc định: "Chưa chín" ---
# Nếu không khớp overripe và ripe → quả còn xanh
u = rules.get("unripe", {})
return RipenessResult(
    u.get("stage", "unripe"),
    u.get("label", "Chưa chín"),
    hue, 0.80
)

def _unknown(self):
    # Trả về khi không đủ điều kiện phân tích (bounding box quá nhỏ, quá
ít pixel hợp lệ)
    return RipenessResult("unknown", "Không rõ", -1.0, 0.0)

```

2.2.7 Bảng quy tắc RIPENESS_RULES cho 10 loại quả

Bảng quy tắc dưới đây được xây dựng dựa trên quan sát thực nghiệm về sự thay đổi màu sắc vỏ quả theo từng giai đoạn chín. Mỗi loại quả có ngưỡng Hue riêng phản ánh đặc điểm màu sắc tự nhiên của nó

```
python
RIPENESS_RULES = {
    # Táo: xanh khi còn xanh lá (H=25-80), đỏ khi chín (H=0-10 và wrap 160-180)
    "apple": {
        "unripe": {"hue": (25, 80), "label": "Chưa chín", "stage": "unripe"},
        "ripe": {"hue": (0, 10), "label": "Đã chín", "stage": "ripe"},
        "ripe_wrap": (160, 180), # Đỏ tươi nằm ở cuối vòng tròn màu
        "overripe": {"hue": (0, 5), "label": "Quá chín", "stage": "overripe"},
        "overripe_wrap": (168, 180), # Đỏ sẫm, gần thối
    },
    # Bơ: xanh lá khi chưa chín, chuyển sang tím/đen khi chín, nâu khi quá chín
    "avocado": {
        "unripe": {"hue": (25, 90), "label": "Chưa chín", "stage": "unripe"},
        "ripe": {"hue": (120, 180), "label": "Đã chín", "stage": "ripe"},
        "ripe_wrap": (0, 20), # Nâu sẫm (vẫn nằm quanh 0°)
        "overripe": {"hue": (0, 5), "label": "Quá chín", "stage": "overripe"},
    },
    # Chuối: xanh (H=28-85), vàng khi chín (H=15-30), nâu khi quá chín (H=0-17)
    "banana": {
        "unripe": {"hue": (28, 85), "label": "Chưa chín", "stage": "unripe"},
        "ripe": {"hue": (15, 30), "label": "Đã chín", "stage": "ripe"},
        "overripe": {"hue": (0, 17), "label": "Quá chín", "stage": "overripe"},
    },
    # Thanh long: xanh khi chưa chín, hồng/đỏ đậm khi chín (H=140-170)
    "dragon fruit": {
        "unripe": {"hue": (28, 80), "label": "Chưa chín", "stage": "unripe"},
        "ripe": {"hue": (140, 170), "label": "Đã chín", "stage": "ripe"},
        "overripe": {"hue": (0, 10), "label": "Quá chín", "stage": "overripe"},
    },
    # Chanh: xanh (H=45-90), vàng sáng khi chín (H=30-45)
    "lemon": {
        "unripe": {"hue": (45, 90), "label": "Chưa chín", "stage": "unripe"},
        "ripe": {"hue": (30, 45), "label": "Đã chín", "stage": "ripe"},
    },
}
```

```
        "overripe": {"hue": (0, 30), "label": "Quá chín", "stage":
"overripe"},
    },
    # Xoài: xanh khi còn non (H=32-85), vàng cam khi chín (H=13-35)
    "mango": {
        "unripe": {"hue": (32, 85), "label": "Chưa chín", "stage":
"unripe"},
        "ripe": {"hue": (13, 35), "label": "Đã chín", "stage": "ripe"},
        "overripe": {"hue": (0, 15), "label": "Quá chín", "stage":
"overripe"},
    },
    # Cam: màu cam thuần (H=8-28), xanh khi còn non
    "orange": {
        "unripe": {"hue": (28, 80), "label": "Chưa chín", "stage":
"unripe"},
        "ripe": {"hue": (8, 28), "label": "Đã chín", "stage":
"ripe"},
        "overripe": {"hue": (0, 10), "label": "Quá chín", "stage":
"overripe"},
        "overripe_wrap": (168, 180),
    },
    # Đu đủ: vàng cam khi chín (H=8-25)
    "papaya": {
        "unripe": {"hue": (28, 80), "label": "Chưa chín", "stage":
"unripe"},
        "ripe": {"hue": (8, 25), "label": "Đã chín", "stage": "ripe"},
        "overripe": {"hue": (0, 10), "label": "Quá chín", "stage":
"overripe"},
    },
    # Dứa: vàng khi chín (H=16-30)
    "pineapple": {
        "unripe": {"hue": (28, 80), "label": "Chưa chín", "stage":
"unripe"},
        "ripe": {"hue": (16, 30), "label": "Đã chín", "stage": "ripe"},
        "overripe": {"hue": (0, 18), "label": "Quá chín", "stage":
"overripe"},
    },
    # Dâu tây: đỏ tươi khi chín (H=0-10 và wrap 160-180), tương tự táo
    "strawberry": {
        "unripe": {"hue": (28, 80), "label": "Chưa chín", "stage":
"unripe"},
        "ripe": {"hue": (0, 10), "label": "Đã chín", "stage":
"ripe"},
        "ripe_wrap": (160, 180),
        "overripe": {"hue": (0, 5), "label": "Quá chín", "stage":
"overripe"},
        "overripe_wrap": (170, 180),
    },
```


}

2.3. Cơ sở dữ liệu dinh dưỡng (nutrition_database.json)

Cơ sở dữ liệu dinh dưỡng được xây dựng theo chuẩn **USDA FoodData Central** - cơ quan quản lý thực phẩm Hoa Kỳ, nguồn dữ liệu dinh dưỡng uy tín và chi tiết nhất hiện nay. File JSON chứa thông tin của **30 loại trái cây**, trong đó 10 loại được model nhận diện và 20 loại còn lại được chuẩn bị sẵn cho việc mở rộng hệ thống sau này.

```
json
{
  "apple_raw": {
    "name_vn": "Táo",
    "energy_kcal": 52,
    "fat_g": 0.17,
    "fiber_g": 2.4,
    "calcium_mg": 6,
    "magnesium_mg": 5,
    "zinc_mg": 0.04,
    "vitamin_a_iu": 54,
    "name_en": "Apple, raw",
    "protein_g": 0.26,
    "carbs_g": 13.81,
    "sugar_g": 10.39,
    "iron_mg": 0.12,
    "potassium_mg": 107,
    "vitamin_c_mg": 4.6,
  },
  "avocado_raw": {
    "name_vn": "Bơ",
    "energy_kcal": 160,
    "fat_g": 14.66,
    "fiber_g": 6.7,
    "calcium_mg": 12,
    "magnesium_mg": 29,
    "zinc_mg": 0.64,
    "vitamin_a_iu": 146,
    "name_en": "Avocado, raw",
    "protein_g": 2.00,
    "carbs_g": 8.53,
    "sugar_g": 0.66,
    "iron_mg": 0.55,
    "potassium_mg": 485,
    "vitamin_c_mg": 10.0,
  }
}
```

Để tra cứu dữ liệu, tên lớp model (ví dụ 'apple') cần được ánh xạ sang key trong JSON (ví dụ 'apple_raw'). Từ điển 'CLASS_TO_NUTRITION_KEY' thực hiện việc này. Lý do hai tên khác nhau là để model sử dụng tên ngắn gọn còn database dùng tên chuẩn USDA:

```
python
CLASS_TO_NUTRITION_KEY = {
  'apple': 'apple_raw',      # Táo
  'avocado': 'avocado_raw',  # Bơ
  'banana': 'banana_raw',    # Chuối
  'dragon fruit': 'dragonfruit_raw', # Thanh long
  'lemon': 'lemon_raw',      # Chanh
  'mango': 'mango_raw',      # Xoài
  'orange': 'orange_raw',    # Cam
  'papaya': 'papaya_raw',    # Đu đủ
  'pineapple': 'pineapple_raw', # Dứa
  'strawberry': 'strawberry_raw', # Dâu tây
}
```

Khi đã có `class\_name` sau heuristic, chỉ cần hai dòng để lấy toàn bộ dữ liệu dinh dưỡng:

```
python
key      = CLASS_TO_NUTRITION_KEY.get(class_name.lower()) # Lấy key tương
ứng
nutrition = _nutrition_db.get(key) if key else None        # Tra cứu trong
dict đã nạp
```

Sau đó dữ liệu dinh dưỡng được ghép vào kết quả cuối cùng và trả về cho Backend:

```
python
if nutrition:
    result["nutrition"] = {
        "name_en":    nutrition.get("name_en", class_name),
        "name_vn":    nutrition.get("name_vn", ""),
        "energy_kcal": nutrition.get("energy_kcal"),      # Năng lượng
(kcal/100g)
        "protein_g":  nutrition.get("protein_g"),       # Đạm
        "fat_g":      nutrition.get("fat_g"),            # Chất béo
        "carbs_g":    nutrition.get("carbs_g"),          # Tinh bột
        "fiber_g":    nutrition.get("fiber_g"),          # Chất xơ
        "sugar_g":    nutrition.get("sugar_g"),          # Đường
        "calcium_mg": nutrition.get("calcium_mg"),       # Canxi
        "iron_mg":    nutrition.get("iron_mg"),          # Sắt
        "magnesium_mg": nutrition.get("magnesium_mg"),  # Magie
        "potassium_mg": nutrition.get("potassium_mg"),  # Kali
        "zinc_mg":    nutrition.get("zinc_mg"),          # Kẽm
        "vitamin_c_mg": nutrition.get("vitamin_c_mg"),  # Vitamin C
        "vitamin_a_iu": nutrition.get("vitamin_a_iu"),  # Vitamin A
    }
else:
    result["nutrition"] = None # Loại quả chưa có trong database
```

2.4. Flask AI Service

Flask được chọn làm web framework cho AI Service vì nhẹ, dễ cấu hình và tương thích tốt với hệ sinh thái Python AI. File `app.py` là entry point của AI Service, chịu trách nhiệm duy nhất là nhận ảnh qua HTTP và trả về kết quả JSON — toàn bộ logic AI nằm trong `model\_service.py`.

Khi khởi tạo, Flask được cấu hình để chấp nhận request từ cả Backend (port 5000) và Frontend (port 5173) thông qua CORS:

```
python
app = Flask(__name__)
# CORS cho phép Frontend và Backend gọi AI Service mà không bị trình duyệt
chặn
CORS(app, origins=["http://localhost:5000", "http://localhost:5173"])
```

```
# Tạo thư mục uploads nếu chưa tồn tại (dùng exist_ok để không báo lỗi nếu đã có)
UPLOAD_DIR = Path(__file__).parent / "uploads"
UPLOAD_DIR.mkdir(exist_ok=True)
```

```
# Danh sách định dạng ảnh được chấp nhận
ALLOWED_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
```

Endpoint `/health` phục vụ mục đích kiểm tra sức khỏe dịch vụ. Backend gọi endpoint này trước khi forward ảnh để đảm bảo AI Service đang chạy:

```
python
@app.route("/health", methods=["GET"])
def health():
    # Trả về trạng thái "ok" để Backend biết AI Service sẵn sàng nhận request
    return jsonify({"status": "ok", "service": "fruit-ai"})
```

Endpoint `/predict` là trung tâm của AI Service. Nó hỗ trợ hai định dạng đầu vào khác nhau trong cùng một hàm để phục vụ cả upload ảnh lẫn nhận diện webcam:

```
python
@app.route("/predict", methods=["POST"])
def predict():
    try:
        image_bytes = None

        # --- Trường hợp 1: Ảnh tải lên dưới dạng file (multipart/form-data) ---
        --
        # Đây là định dạng chuẩn khi người dùng upload file qua form HTML
        if "image" in request.files:
            file = request.files["image"]
            # Lấy phần mở rộng file và chuyển về chữ thường để so sánh
            ext = Path(file.filename).suffix.lower() if file.filename else
            ".jpg"

            if ext not in ALLOWED_EXTENSIONS:
                # Từ chối ngay nếu không phải định dạng ảnh được hỗ trợ
                return jsonify({"success": False,
                                "message": f"Unsupported file type: {ext}"}), 400
            image_bytes = file.read() # Đọc toàn bộ nội dung file vào RAM

        # --- Trường hợp 2: Ảnh webcam dưới dạng chuỗi Base64 trong JSON ---
        # Trình duyệt gửi ảnh chụp từ webcam dưới dạng:
        "data:image/jpeg;base64,/9j/4AA..."
        elif request.is_json:
            data = request.get_json()
            b64data = data.get("image_base64", "")
            if not b64data:
                return jsonify({"success": False, "message": "No image data
provided."}), 400
            import base64
            # Cắt bỏ phần tiền tố "data:image/jpeg;base64," nếu có
            if "," in b64data:
```

```

        b64data = b64data.split(",", 1)[1]
        # Giải mã Base64 → bytes thô của ảnh JPEG
        image_bytes = base64.b64decode(b64data)

    else:
        return jsonify({"success": False,
                        "message": "No image provided. Use multipart or base64
JSON."}), 400

    # Gọi hàm nhận diện từ model_service.py
    result = predict_image(image_bytes)
    return jsonify(result)

except ValueError as e:
    # ValueError được raise khi: ảnh không giải mã được, không tìm thấy
    quả
    return jsonify({"success": False, "message": str(e)}), 422
except Exception as e:
    # Bắt mọi lỗi không mong đợi, ghi log chi tiết, không để lộ stack
    trace cho client
    app.logger.error(f"Prediction error: {e}", exc_info=True)
    return jsonify({"success": False,
                    "message": "Internal server error during prediction."}), 500

```

Cuối cùng, khi khởi động service, model được nạp vào RAM trước khi Flask bắt đầu nhận request. Điều này đảm bảo request đầu tiên không phải chờ thêm thời gian nạp model:

2.5. Node.js Backend — Điều phối nghiệp vụ

2.5.1. Entry point và cấu hình server

- Backend Node.js đóng vai trò trung tâm: tiếp nhận request từ Frontend, xác thực người dùng, forward ảnh sang AI Service, lưu kết quả vào MySQL và phát sự kiện realtime qua Socket.IO. File `server.js` thiết lập toàn bộ hạ tầng này.
- Điểm đặc biệt là `http.createServer(app)` — thay vì để Express tự tạo server, ta tạo server HTTP thủ công để Socket.IO có thể gắn vào cùng một cổng với REST API, tránh phải mở thêm cổng mạng:

```

javascript
const app    = express();
const server = http.createServer(app); // Tạo HTTP server từ Express app
const io     = new Server(server, {   // Gắn Socket.IO vào cùng HTTP server
  cors: {
    origin: ['http://localhost:5173', 'http://localhost:3000'],
    methods: ['GET', 'POST']
  }
});

// Đặt instance io vào app context để các controller có thể dùng mà không cần
import
app.set('io', io);

```

Tiếp theo cấu hình các middleware cần thiết. Giới hạn body size được nâng lên 50MB để hỗ trợ ảnh base64 từ webcam (ảnh JPEG 1MB sau khi encode Base64 tăng lên ~1.33MB):

```
javascript
app.use(cors({ origin: ['http://localhost:5173', 'http://localhost:3000'],
credentials: true }));
app.use(express.json({ limit: '50mb' })); // Parse JSON body, cho
phép ảnh base64 lớn
app.use(express.urlencoded({ extended: true, limit: '50mb' }));

// Phục vụ file ảnh đã upload tại URL /uploads/filename.jpg
app.use('/uploads', express.static(path.join(__dirname, '../uploads')));
Tất cả route được tổ chức theo nhóm chức năng và mount vào tiền tố URL tương ứng:
```

```
javascript
app.use('/api/auth',      require('./routes/auth')); // Đăng ký, đăng
nhập, profile
app.use('/api/predictions', require('./routes/predictions')); // Nhận diện ảnh
+ webcam
app.use('/api/history',    require('./routes/history')); // Lịch sử +
xuất Excel/PDF
app.use('/api/dashboard',  require('./routes/dashboard')); // Thống kê biểu
đồ
app.use('/api/admin',      require('./routes/admin')); // Quản trị hệ
thống (admin only)

// Endpoint kiểm tra sức khỏe Backend
app.get('/api/health', (req, res) =>
  res.json({ status: 'ok', timestamp: new Date().toISOString() })
);
```

2.5.2. Socket.IO — Thông báo realtime

Socket.IO cho phép Backend đẩy dữ liệu mới xuống Frontend ngay lập tức mà không cần Frontend phải polling. Tuy nhiên, WebSocket cần xác thực riêng vì không có HTTP header thông thường. Middleware dưới đây xác thực JWT ngay tại thời điểm kết nối WebSocket:

```
javascript

// Middleware chạy khi có kết nối WebSocket mới, TRƯỚC khi kết nối được chấp
nhận
io.use((socket, next) => {
  const token = socket.handshake.auth?.token; // Frontend gửi token qua auth
object
  if (!token) return next(new Error('Authentication required'));
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    socket.user = decoded; // Gắn thông tin user vào socket để dùng sau
    next(); // Chấp nhận kết nối
  } catch {
```

```

    next(new Error('Invalid token')); // Từ chối kết nối nếu token không hợp
    lệ
  }
});

// Sau khi kết nối được chấp nhận
io.on('connection', socket => {
  const userId = socket.user?.id;
  if (userId) {
    // Mỗi user join vào một "room" riêng có tên 'user_{id}'
    // Nhờ đó Backend có thể gửi sự kiện chỉ đến đúng user đó, không broadcast
    toàn bộ
    socket.join(`user_${userId}`);
    console.log(`[WS] User ${userId} connected (socket: ${socket.id})`);
  }
  socket.on('disconnect', () => {
    console.log(`[WS] Socket ${socket.id} disconnected`);
  });
});

```

2.5.3. Middleware xác thực JWT

File `middleware/auth.js` định nghĩa hai middleware phân quyền. Chúng được áp dụng như "người gác cổng" trên mỗi route cần bảo vệ.

`authMiddleware` kiểm tra JWT trong header `Authorization: Bearer <token>`. Nếu hợp lệ, thông tin user được gắn vào `req.user` để controller sử dụng. Nếu không hợp lệ, request bị chặn ngay tại đây với mã lỗi 401:

```

javascript
function authMiddleware(req, res, next) {
  const header = req.headers.authorization || '';
  // Token phải có dạng "Bearer eyJ..." - kiểm tra tiền tố
  if (!header.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, message: 'No token
provided.' });
  }
  const token = header.slice(7); // Cắt 7 ký tự "Bearer " ở đầu
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, name, email, role } - dùng được ở mọi
controller sau
    next(); // Tiếp tục xử lý request
  } catch (err) {
    // Token hết hạn hoặc bị giả mạo → từ chối
    return res.status(401).json({ success: false, message: 'Invalid or expired
token.' });
  }
}

```

`adminMiddleware` được dùng thêm SAU `authMiddleware` cho các route chỉ admin được truy cập. Nó chỉ kiểm tra trường `role` trong thông tin user đã xác thực:

```
javascript
function adminMiddleware(req, res, next) {
  // req.user đã được gắn bởi authMiddleware trước đó
  if (req.user?.role !== 'admin') {
    return res.status(403).json({ success: false, message: 'Admin access
required.' });
  }
  next(); // Qua được kiểm tra → cho phép tiếp tục
}
```

2.5.4. Định nghĩa routes

Auth routes (`routes/auth.js`) — Bốn endpoint phục vụ toàn bộ vòng đời tài khoản người dùng. Các route không cần đăng nhập (`register`, `login`) không gắn middleware. Các route cần bảo vệ (`me`, `profile`) thêm `authMiddleware`:

```
javascript
router.post('/register', ctrl.register); // Tạo tài khoản mới
router.post('/login', ctrl.login); // Đăng nhập → nhận JWT
router.get('/me', authMiddleware, ctrl.getMe); // Lấy thông tin user
đang đăng nhập
router.put('/profile', authMiddleware, ctrl.updateProfile); // Cập nhật
tên/mật khẩu
```

Predictions routes (`routes/predictions.js`) - Sử dụng Multer để xử lý upload file. Ảnh được giữ trong RAM (memoryStorage) thay vì lưu ngay ra disk, vì ta cần forward bytes sang AI Service trước. Chỉ sau khi AI phân tích thành công mới lưu file:

```
javascript
// Cấu hình Multer: lưu file vào bộ nhớ RAM (buffer)
const upload = multer({
  storage: multer.memoryStorage(),
  // MAX_FILE_SIZE đọc từ .env, mặc định 10MB (10 * 1024 * 1024 bytes)
  limits: { fileSize: parseInt(process.env.MAX_FILE_SIZE || '10485760') },
  fileFilter: (req, file, cb) => {
    const allowed = ['image/jpeg', 'image/png', 'image/webp', 'image/bmp'];
    // Kiểm tra MIME type thay vì extension — chính xác hơn vì dựa trên nội
dung file
    if (allowed.includes(file.mimetype)) cb(null, true);
    else cb(new Error('Only JPEG, PNG, WebP, BMP images are allowed.'));
  },
});
/ upload.single('image'): chỉ nhận đúng 1 file với field name là 'image'
router.post('/upload', authMiddleware, upload.single('image'),
ctrl.uploadPredict);
// Webcam không cần Multer vì ảnh đến dưới dạng JSON base64
router.post('/webcam', authMiddleware, ctrl.webcamPredict);
```

History routes** (`routes/history.js`) — Lưu ý quan trọng: `/export/excel` và `/export/pdf` phải được khai báo **trước** `/:id`. Nếu đặt sau, Express sẽ nhầm "excel" và "pdf" là giá trị của param `:id`:

```
javascript
router.get('/export/excel', authMiddleware, ctrl.exportExcel); // Phải đứng
trước /:id
```

```
router.get('/export/pdf', authMiddleware, ctrl.exportPDF); // Phải đứng
trước /:id
router.get('/', authMiddleware, ctrl.getHistory);
router.get('/:id', authMiddleware, ctrl.getOne);
router.delete('/', authMiddleware, ctrl.deleteAll);
router.delete('/:id', authMiddleware, ctrl.deleteOne);
```

Admin routes (`routes/admin.js`) — Tất cả route admin đều yêu cầu cả hai middleware. Dùng spread operator `...guard` để viết gọn, dễ đọc:

```
javascript
const guard = [authMiddleware, adminMiddleware]; // Kết hợp hai middleware
thành mảng

router.get('/users', ...guard, ctrl.getAllUsers); // Danh sách
user
router.delete('/users/:id', ...guard, ctrl.deleteUser); // Xóa user
router.get('/records', ...guard, ctrl.getAllRecords); // Tất cả bản
ghi
router.delete('/records/:id', ...guard, ctrl.deleteRecord); // Xóa bản ghi
router.get('/stats', ...guard, ctrl.getSystemStats); // Thống kê hệ
thống
```

2.6. Controllers — Cài đặt chi tiết logic nghiệp vụ

2.6.1. authController.js — Xác thực người dùng

Hàm helper `signToken()` tập trung việc tạo JWT vào một nơi. Payload token chứa đủ thông tin để middleware giải mã mà không cần query database lại:

```
javascript
function signToken(user) {
  return jwt.sign(
    // Payload: thông tin cần có trong token để xác thực và phân quyền
    { id: user.id, name: user.name, email: user.email, role: user.role },
    process.env.JWT_SECRET, // Khóa bí mật ký token
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' } // Token hết hạn sau 7
ngày
  );
}
```

Luồng đăng ký kiểm tra đầy đủ: validate input, kiểm tra email trùng, hash mật khẩu rồi lưu vào DB. **Bcrypt** với cost factor 10 tạo ra hash không thể đảo ngược, đảm bảo dù database bị lộ, mật khẩu thật vẫn an toàn:

```
javascript
exports.register = async (req, res) => {
  const { name, email, password } = req.body;
  // Validate dữ liệu đầu vào trước khi xử lý
  if (!name || !email || !password)
    return res.status(400).json({ success: false, message: 'All fields are
required.' });
  if (password.length < 6)
```



```
    return res.status(400).json({ success: false, message: 'Password must be
at least 6 characters.' });

    // Kiểm tra email tồn tại. Dùng parameterized query (?) để tránh SQL
Injection
    const [existing] = await db.query('SELECT id FROM users WHERE email = ?',
[email]);
    if (existing.length > 0)
        return res.status(409).json({ success: false, message: 'Email already
registered.' });

    // Hash mật khẩu với bcrypt, cost factor 10: tạo ra chuỗi 60 ký tự không thể
giải mã ngược
    const hashed = await bcrypt.hash(password, 10);
    const [result] = await db.query(
        'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
        [name.trim(), email.toLowerCase().trim(), hashed] // Chuẩn hóa: trim +
lowercase email
    );

    const user = { id: result.insertId, name: name.trim(), email:
email.toLowerCase().trim(), role: 'user' };
    const token = signToken(user);
    res.status(201).json({ success: true, token, user }); // 201 Created
};
```

Luồng đăng nhập dùng `bcrypt.compare()` — hàm này tự động tái tạo hash từ password thật và so sánh với hash trong DB, không cần lưu trữ password thật ở bất kỳ đâu:

```
javascript
exports.login = async (req, res) => {
    const { email, password } = req.body;
    const [rows] = await db.query('SELECT * FROM users WHERE email = ?',
[email.toLowerCase().trim()]);
    // Dùng cùng một thông báo lỗi "Invalid credentials" dù email sai hay mật
khẩu sai
    // → tránh hacker biết được email nào tồn tại trong hệ thống
    if (rows.length === 0)
        return res.status(401).json({ success: false, message: 'Invalid
credentials.' });

    const user = rows[0];
    const valid = await bcrypt.compare(password, user.password); // So sánh
hash
    if (!valid)
        return res.status(401).json({ success: false, message: 'Invalid
credentials.' });

    const token = signToken(user);
```

```
// Destructuring để loại bỏ trường 'password' khỏi object trước khi trả về
// Không bao giờ gửi hash mật khẩu về client
const { password: _, ...safeUser } = user;
res.json({ success: true, token, user: safeUser });
});
```

Cập nhật hồ sơ hỗ trợ đổi tên và/hoặc đổi mật khẩu. Câu SQL UPDATE được xây dựng động từ chỉ những trường thực sự thay đổi:

```
javascript
exports.updateProfile = async (req, res) => {
  const { name, currentPassword, newPassword } = req.body;
  const [rows] = await db.query('SELECT * FROM users WHERE id = ?',
[req.user.id]);
  const user = rows[0];
  const updates = {}; // Object chứa các cột cần UPDATE

  if (name) updates.name = name.trim();

  if (newPassword) {
    // Bắt buộc phải có mật khẩu cũ để xác nhận danh tính trước khi đổi mới
    if (!currentPassword)
      return res.status(400).json({ success: false, message: 'Current password
required.' });
    const valid = await bcrypt.compare(currentPassword, user.password);
    if (!valid)
      return res.status(401).json({ success: false, message: 'Current password
incorrect.' });
    if (newPassword.length < 6)
      return res.status(400).json({ success: false, message: 'New password
must be at least 6 characters.' });
    updates.password = await bcrypt.hash(newPassword, 10);
  }

  if (Object.keys(updates).length === 0)
    return res.status(400).json({ success: false, message: 'Nothing to
update.' });

  // Xây dựng "name = ?, email = ?" động từ keys của object updates
  const setClauses = Object.keys(updates).map(k => `${k} = ?`).join(', ');
  await db.query(`UPDATE users SET ${setClauses} WHERE id = ?`,
[...Object.values(updates), req.user.id]);

  // Trả về thông tin user mới nhất (không có password)
  const [updated] = await db.query(
'SELECT id, name, email, role, avatar, created_at FROM users WHERE id =
?', [req.user.id]);
  res.json({ success: true, user: updated[0] });
};
```

2.6.2. predictionController.js — Luồng nhận diện chính

Hàm `callAIService()` chịu trách nhiệm forward ảnh từ Node.js sang Python Flask. Ảnh được đóng gói dưới dạng `multipart/form-data` - giống như khi người dùng upload file thông thường:

```
javascript
async function callAIService(imageBuffer, mimeType = 'image/jpeg') {
  const form = new FormData();
  // Thêm ảnh vào form với field name là 'image' - Flask sẽ đọc theo tên này
  form.append('image', imageBuffer, { filename: 'upload.jpg', contentType:
mimeType });

  const response = await fetch(`${AI_URL}/predict`, { method: 'POST', body:
form });
  if (!response.ok) {
    const text = await response.text();
    throw new Error(`AI service error ${response.status}: ${text}`);
  }
  return response.json(); // Parse JSON response từ Flask
}
```

Hàm `savePrediction()` lưu kết quả vào MySQL. Điểm đáng chú ý là các vi lượng (vitamin, khoáng chất) được serialized thành JSON string và lưu vào cột `vitamins` kiểu JSON — thiết kế này cho phép thêm/bớt trường dinh dưỡng mà không cần ALTER TABLE:

```
javascript
async function savePrediction(userId, imagePath, aiResult) {
  const n = aiResult.nutrition || {};
  // Gộp các vi lượng vào một JSON string để lưu vào cột vitamins
  const vitaminsJson = JSON.stringify({
    vitamin_c_mg: n.vitamin_c_mg ?? null, // ?? null: dùng null nếu giá trị
undefined
    vitamin_a_iu: n.vitamin_a_iu ?? null,
    calcium_mg: n.calcium_mg ?? null,
    iron_mg: n.iron_mg ?? null,
    magnesium_mg: n.magnesium_mg ?? null,
    potassium_mg: n.potassium_mg ?? null,
    zinc_mg: n.zinc_mg ?? null,
    fiber_g: n.fiber_g ?? null,
    sugar_g: n.sugar_g ?? null,
  });

  const [result] = await db.query(
    `INSERT INTO predictions
      (user_id, image_path, fruit_name, confidence, ripeness,
       calories, carbohydrates, protein, fat, vitamins)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    [userId, imagePath, aiResult.fruit_name, aiResult.confidence,
aiResult.ripeness || null, n.energy_kcal ?? null, n.carbs_g ?? null,
n.protein_g ?? null, n.fat_g ?? null, vitaminsJson]
```

```
);
return result.insertId; // Trả về ID của bản ghi vừa tạo
}
```

Controller `uploadPredict` xử lý ảnh tải lên theo trình tự: xác thực → gọi AI → lưu file → lưu DB → thông báo realtime → trả kết quả:

```
javascript
exports.uploadPredict = async (req, res) => {
  if (!req.file) // Multer đặt file vào req.file nếu upload thành công
    return res.status(400).json({ success: false, message: 'No image
uploaded.' });

  const imageBuffer = req.file.buffer; // Buffer chứa dữ liệu ảnh trong RAM
  const mimeType = req.file.mimetype;
  const aiResult = await callAIService(imageBuffer, mimeType);

  if (!aiResult.success)
    return res.status(422).json({ success: false, message: aiResult.message
});

  // Chỉ lưu file ra disk sau khi AI xử lý thành công
  // Tên file: timestamp + tên gốc đã làm sạch ký tự đặc biệt
  const filename = `${Date.now()}_${req.file.originalname.replace(/[^\a-zA-Z0-9.]/g, '_')}';
  const uploadDir = path.join(__dirname, '../..../uploads');
  if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });
  fs.writeFileSync(path.join(uploadDir, filename), imageBuffer);

  const predId = await savePrediction(req.user.id, `/uploads/${filename}`,
aiResult);

  // JOIN với bảng users để lấy user_name cho hiển thị trong lịch sử
  const [rows] = await db.query(
    `SELECT p.*, u.name as user_name FROM predictions p
      JOIN users u ON p.user_id = u.id WHERE p.id = ?`, [predId]);
  const record = rows[0];

  // Phát sự kiện Socket.IO đến đúng room của user này
  // Frontend đang listen 'new_prediction' → tự động cập nhật danh sách lịch
sử
  const io = req.app.get('io');
  if (io) io.to(`user_${req.user.id}`).emit('new_prediction', record);

  res.json({ success: true, prediction: record });
};
```

2.6.3. historyController.js — Quản lý lịch sử

Hàm `getHistory()` xây dựng câu SQL động để hỗ trợ tìm kiếm, lọc ngày và phân trang cùng lúc. Thay vì viết nhiều câu SQL riêng, một câu SQL với WHERE clause động phục vụ tất cả các trường hợp:

```
javascript
exports.getHistory = async (req, res) => {
  const { page = 1, limit = 10, search = '', startDate, endDate } = req.query;

  const offset = (parseInt(page) - 1) * parseInt(limit); // Tính vị trí bắt đầu
  const params = [req.user.id]; // Mảng tham số cho parameterized query
  let whereClause = 'WHERE user_id = ?'; // Luôn lọc theo user hiện tại

  // Thêm điều kiện tìm kiếm tên quả nếu có
  if (search) {
    whereClause += ' AND fruit_name LIKE ?';
    params.push(`%${search}%`); // % là ký tự wildcard trong SQL LIKE
  }

  // Thêm lọc ngày bắt đầu nếu có
  if (startDate) {
    whereClause += ' AND DATE(created_at) >= ?';
    params.push(startDate);
  }

  // Thêm lọc ngày kết thúc nếu có
  if (endDate) {
    whereClause += ' AND DATE(created_at) <= ?';
    params.push(endDate);
  }

  // Query 1: đếm tổng số bản ghi thỏa điều kiện (dùng để tính số trang)
  const [countResult] = await db.query(
    `SELECT COUNT(*) as total FROM predictions ${whereClause}`, params);
  const total = countResult[0].total;

  // Query 2: lấy dữ liệu của trang hiện tại với LIMIT và OFFSET
  const [rows] = await db.query(
    `SELECT * FROM predictions ${whereClause}
    ORDER BY created_at DESC LIMIT ? OFFSET ?`,
    [...params, parseInt(limit), offset]
  );

  // Trả về cả dữ liệu lẫn thông tin phân trang để Frontend render pagination
  res.json({
    success: true,
    data: rows,
    pagination: {
      total,
      page: parseInt(page),
      limit: parseInt(limit),
      totalPages: Math.ceil(total / parseInt(limit)),
    }
  });
}
```

```

    },
  });
};

```

Khi xóa bản ghi, file ảnh vật lý trên disk cũng được xóa để tránh tích lũy file rác. Logic xóa áp dụng cả khi xóa một bản ghi lẫn xóa tất cả:

```

javascript
exports.deleteOne = async (req, res) => {
  // Xác minh bản ghi thuộc về user hiện tại trước khi xóa
  const [rows] = await db.query(
    'SELECT * FROM predictions WHERE id = ? AND user_id = ?',
    [req.params.id, req.user.id]);
  if (rows.length === 0)
    return res.status(404).json({ success: false, message: 'Record not found.' });

  // Tạo đường dẫn tuyệt đối đến file ảnh và xóa nếu tồn tại
  const imgPath = path.join(__dirname, '../..', rows[0].image_path);
  if (fs.existsSync(imgPath)) fs.unlinkSync(imgPath);

  await db.query('DELETE FROM predictions WHERE id = ? AND user_id = ?',
    [req.params.id, req.user.id]);
  res.json({ success: true, message: 'Deleted successfully.' });
};

```

Chức năng xuất Excel dùng thư viện **ExcelJS** để tạo file `.xlsx` có định dạng: header màu xanh lá, dữ liệu từng hàng tương ứng với các cột đã định nghĩa. File được stream trực tiếp về client mà không cần lưu ra disk:

```

javascript
exports.exportExcel = async (req, res) => {
  const ExcelJS = require('exceljs');
  const [rows] = await db.query(
    'SELECT * FROM predictions WHERE user_id = ? ORDER BY created_at DESC',
    [req.user.id]);

  const workbook = new ExcelJS.Workbook();
  const worksheet = workbook.addWorksheet('Fruit History');

  // Định nghĩa các cột: header hiển thị, key lấy từ object row, độ rộng cột
  worksheet.columns = [
    { header: 'ID', key: 'id', width: 8 },
    { header: 'Fruit', key: 'fruit_name', width: 15 },
    { header: 'Confidence (%)', key: 'confidence', width: 16 },
    { header: 'Ripeness', key: 'ripeness', width: 15 },
    { header: 'Calories', key: 'calories', width: 12 },
    { header: 'Protein (g)', key: 'protein', width: 13 },
    { header: 'Fat (g)', key: 'fat', width: 10 },
    { header: 'Carbs (g)', key: 'carbohydrates', width: 13 },
  ];

```

```

    { header: 'Date',          key: 'created_at',    width: 22 },
  ];

  // Tô màu xanh lá (#10b981) và chữ trắng, in đậm cho hàng header (hàng 1)
  worksheet.getRow(1).eachCell(cell => {
    cell.fill = { type: 'pattern', pattern: 'solid', fgColor: { argb:
'FF10b981' } };
    cell.font = { color: { argb: 'FFFFFF' }, bold: true };
  });

  rows.forEach(row => worksheet.addRow(row)); // Thêm từng hàng dữ liệu

  // Thiết lập Content-Type để trình duyệt tự động tải về thay vì hiển thị
  res.setHeader('Content-Type', 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet');
  res.setHeader('Content-Disposition', 'attachment;
filename=fruit_history.xlsx');
  await workbook.xlsx.write(res); // Stream file trực tiếp vào response
  res.end();
};

```

Xuất PDF dùng **jsPDF** với plugin `autoTable` để tạo bảng có định dạng tự động. File được tạo ở chế độ landscape (ngang) để đủ chỗ cho nhiều cột:

```

javascript
exports.exportPDF = async (req, res) => {
  const { jsPDF } = require('jspdf');
  require('jspdf-autotable'); // Plugin tạo bảng tự động cho jsPDF

  const [rows] = await db.query(
    'SELECT * FROM predictions WHERE user_id = ? ORDER BY created_at DESC',
    [req.user.id]);

  const doc = new jsPDF({ orientation: 'landscape' }); // Chế độ nằm ngang
  doc.setFontSize(16);
  doc.text('Fruit Recognition History', 14, 15); // Tiêu đề ở tọa độ (14, 15)
  doc.setFontSize(10);
  doc.text(`Generated: ${new Date().toLocaleString()}`, 14, 22); // Ngày tạo

  // autoTable tự động chia hàng khi nội dung dài hơn chiều cao trang
  doc.autoTable({
    startY: 28, // Bắt đầu bảng tại y=28, sau phần tiêu đề
    head: [['Fruit', 'Confidence', 'Ripeness', 'Calories', 'Protein', 'Fat',
'Carbs', 'Date']],
    body: rows.map(r => [
      r.fruit_name,
      `${r.confidence}%`,
      r.ripeness || '-',
      r.calories      ? `${r.calories} kcal` : '-',
      r.protein       ? `${r.protein} g`    : '-',
    ])
  });
}

```

```

    r.fat          ? `${r.fat} g`      : '-',
    r.carbohydrates ? `${r.carbohydrates} g` : '-',
    new Date(r.created_at).toLocaleDateString(),
  ]),
  styles: { fontSize: 8 },
  headStyles: { fillColor: [16, 185, 129] }, // Màu xanh #10b981 dưới dạng
  RGB
});

const buffer = Buffer.from(doc.output('arraybuffer'));
res.setHeader('Content-Type', 'application/pdf');
res.setHeader('Content-Disposition', 'attachment;
filename=fruit_history.pdf');
res.send(buffer);
};

```

2.6.4. dashboardController.js — Thống kê và biểu đồ

Các hàm trong dashboard controller thực hiện nhiều câu query SQL tổng hợp để cung cấp số liệu cho các biểu đồ trên Frontend. Đặc biệt, `getWeekly()` và `getMonthly()` có bước "fill" quan trọng: nếu một ngày/tháng không có dữ liệu trong DB, hệ thống vẫn trả về giá trị 0 để biểu đồ không bị gián đoạn:

```

javascript
exports.getWeekly = async (req, res) => {
  // Lấy số lần nhận diện theo ngày trong 7 ngày gần nhất
  const [rows] = await db.query(
    `SELECT DATE(created_at) as date, COUNT(*) as count
    FROM predictions
    WHERE user_id = ? AND created_at >= DATE_SUB(CURDATE(), INTERVAL 6 DAY)
    GROUP BY DATE(created_at)
    ORDER BY date ASC`,
    [req.user.id]
  );

  // Bước fill: tạo mảng 7 phần tử liên tiếp, điền 0 cho ngày không có dữ liệu
  // Đảm bảo biểu đồ luôn hiển thị đủ 7 điểm, không bị thiếu
  const result = [];
  for (let i = 6; i >= 0; i--) {
    const d = new Date();
    d.setDate(d.getDate() - i);
    const dateStr = d.toISOString().slice(0, 10); // Định dạng 'YYYY-MM-DD'
    const found = rows.find(r => r.date.toISOString().slice(0, 10) ===
dateStr);
    result.push({ date: dateStr, count: found ? found.count : 0 });
  }
  res.json({ success: true, data: result });
};

```

2.6.5. adminController.js — Quản trị hệ thống

Admin controller cung cấp khả năng giám sát và quản lý toàn bộ hệ thống. Điểm đặc biệt trong `getAllUsers()` là dùng **correlated subquery** để đếm số lần nhận diện của mỗi user ngay trong câu SELECT, tránh phải query thêm vòng lặp:

```
javascript
exports.getAllUsers = async (req, res) => {
  const [users] = await db.query(
    `SELECT id, name, email, role, created_at,
      -- Subquery tính số bản ghi predictions của từng user
      -- Chạy một lần cho mỗi hàng user → hiển thị prediction_count ngay
      trong bảng
      (SELECT COUNT(*) FROM predictions WHERE user_id = users.id) as
      prediction_count
    FROM users ${where} ORDER BY created_at DESC LIMIT ? OFFSET ?`,
    [...params, parseInt(limit), offset]
  );
  // ...
};
```

Khi admin xóa user, toàn bộ ảnh liên quan cũng được dọn dẹp. Câu `ON DELETE CASCADE` trong schema đảm bảo các bản ghi trong bảng `predictions` tự xóa theo khi user bị xóa:

```
javascript
exports.deleteUser = async (req, res) => {
  // Ngăn admin tự xóa chính mình
  if (parseInt(req.params.id) === req.user.id)
    return res.status(400).json({ success: false, message: 'Cannot delete yourself.' });

  // Lấy danh sách file ảnh của user trước khi xóa (sau khi xóa sẽ không còn records)
  const [rows] = await db.query('SELECT * FROM predictions WHERE user_id = ?', [req.params.id]);
  rows.forEach(row => {
    const imagePath = path.join(__dirname, '../..', row.image_path);
    if (fs.existsSync(imagePath)) fs.unlinkSync(imagePath); // Xóa từng file ảnh
  });

  // Xóa user → nhờ ON DELETE CASCADE, toàn bộ predictions của user tự xóa theo
  await db.query('DELETE FROM users WHERE id = ?', [req.params.id]);
  res.json({ success: true, message: 'User deleted.' });
};
```

2.7. Cơ sở dữ liệu MySQL

2.7.1. Schema thiết kế và lý do các lựa chọn kỹ thuật

Cơ sở dữ liệu gồm hai bảng chính có quan hệ một-nhiều (1 user : N predictions). Toàn bộ schema được lưu trong `database/schema.sql` để có thể khởi tạo lại dễ dàng:

```
sql
CREATE DATABASE IF NOT EXISTS fruit_recognition_db
```

```

    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
-- utf8mb4: hỗ trợ toàn bộ Unicode, kể cả emoji và ký tự tiếng Việt có dấu
-- Nếu dùng utf8 thông thường, các ký tự 4 byte (emoji) sẽ bị lỗi

USE fruit_recognition_db;

CREATE TABLE IF NOT EXISTS users (
    id          INT          AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(150) NOT NULL UNIQUE,    -- UNIQUE: mỗi email chỉ đăng ký
1 lần
    password    VARCHAR(255) NOT NULL,          -- Lưu bcrypt hash, luôn dài 60
ký tự
    role        ENUM('user', 'admin') NOT NULL DEFAULT 'user',
    avatar      VARCHAR(500) NULL,
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    INDEX idx_users_email (email),    -- Index tăng tốc tra cứu theo email khi
đăng nhập
    INDEX idx_users_role (role)      -- Index tăng tốc lọc danh sách admin
) ENGINE=InnoDB;
-- InnoDB: hỗ trợ transaction, foreign key, và ACID – bắt buộc dùng cho
production

CREATE TABLE IF NOT EXISTS predictions (
    id          INT          AUTO_INCREMENT PRIMARY KEY,
    user_id     INT          NOT NULL,
    image_path  VARCHAR(500) NOT NULL,
    fruit_name  VARCHAR(100) NOT NULL,
    confidence  DECIMAL(5,2) NOT NULL,    -- VD: 87.34% – lưu 5 chữ số, 2 sau
dấu phẩy
    ripeness    VARCHAR(50)  NULL,        -- 'unripe' | 'ripe' | 'overripe' |
NULL
    calories    DECIMAL(8,2) NULL,
    carbohydrates DECIMAL(8,2) NULL,
    protein     DECIMAL(8,2) NULL,
    fat         DECIMAL(8,2) NULL,
    fiber       DECIMAL(8,2) NULL,
    sugar       DECIMAL(8,2) NULL,
    vitamins    JSON         NULL,    -- Kiểu JSON: linh hoạt, dễ mở rộng không
cần ALTER TABLE
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    -- ON DELETE CASCADE: khi user bị xóa, toàn bộ predictions của họ tự xóa
theo
    INDEX idx_pred_user_id (user_id),    -- Tăng tốc query theo user
    INDEX idx_pred_fruit_name (fruit_name), -- Tăng tốc tìm kiếm theo tên quả

```

```

INDEX idx_pred_created_at (created_at)    -- Tăng tốc lọc theo ngày tháng
) ENGINE=InnoDB;

-- Tài khoản admin mặc định – INSERT IGNORE để không báo lỗi nếu đã tồn tại
-- Mật khẩu 'admin123' đã được hash trước bằng bcrypt
INSERT IGNORE INTO users (name, email, password, role)
VALUES ('Administrator', 'admin@admin.com',
        '$2b$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', 'admin');
```

2.7.2. Connection Pool — Kết nối hiệu quả với MySQL

File `config/db.js` thiết lập **connection pool** thay vì single connection. Với single connection, nếu hai request đến cùng lúc, request thứ hai phải chờ request đầu xong mới được xử lý. Connection pool duy trì nhiều kết nối song song, giải quyết vấn đề này:

```

javascript
const pool = mysql.createPool({
  host:      process.env.DB_HOST      || 'localhost',
  port:      parseInt(process.env.DB_PORT || '3306'),
  user:      process.env.DB_USER      || 'root',
  password:  process.env.DB_PASSWORD  || 'root',
  database:  process.env.DB_NAME      || 'fruit_recognition_db',
  waitForConnections: true,    // Nếu tất cả 10 connection đang bận → request
                                chờ
  connectionLimit: 10,        // Tối đa 10 kết nối song song cùng lúc
  queueLimit: 0,              // 0 = không giới hạn số request trong hàng đợi
  charset:    'utf8mb4',
});
// Kiểm tra kết nối ngay khi server khởi động
pool.getConnection()
  .then(conn => {
    console.log('[DB] ✓ MySQL connected successfully.');


conn.release(); // Quan trọng: trả kết nối về pool ngay sau khi kiểm tra
xong
  })
  .catch(err => {
    console.error('[DB] ✗ MySQL connection failed:', err.message);
    process.exit(1); // Thoát hoàn toàn nếu không kết nối được DB – không cho
server chạy nữa vời
  });


```

2.8. Biến môi trường và script khởi động

2.8.1. File .env

Mọi thông tin nhạy cảm (mật khẩu DB, khóa JWT) đều được tách ra khỏi code vào file `.env`, không được commit lên Git:

```

ini
NODE_ENV=development
PORT=5000
```

```
# Thông tin kết nối MySQL
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=123456
DB_NAME=fruit_recognition_db

# Khóa bí mật ký JWT – phải thay bằng chuỗi ngẫu nhiên mạnh trong production
JWT_SECRET=your_super_secret_jwt_key_change_in_production_abc123xyz
JWT_EXPIRES_IN=7d    # Token hết hạn sau 7 ngày

# URL nội bộ đến AI Service
AI_SERVICE_URL=http://localhost:5001

MAX_FILE_SIZE=10485760 # 10MB = 10 × 1024 × 1024 bytes
```

2.8.2. Script start_all.ps1

Script PowerShell tự động hóa việc khởi động cả 3 dịch vụ, mỗi dịch vụ trong một cửa sổ terminal riêng. Lưu ý 2 giây delay giữa các dịch vụ: AI Service cần thời gian nạp model (~3-5s) trước khi Backend có thể gọi đến nó:

```
powershell
# Khởi động AI Service – dùng đường dẫn Python trực tiếp từ môi trường
Anaconda tdenv
# Không dùng "conda activate" vì không hoạt động trong subprocess
Start-Process powershell -ArgumentList `
    "-NoExit -Command `\"cd d:\aicode\ai_service; `
    & 'C:/Users/OS/anaconda3/envs/tdenv/python.exe' app.py`\"" `
    -WindowStyle Normal

Start-Sleep -Seconds 2 # Đợi AI Service nạp model YOLOv8 xong

# Khởi động Backend Node.js
Start-Process powershell -ArgumentList `
    "-NoExit -Command `\"cd d:\aicode\backend; npm run dev`\"" -WindowStyle Normal

Start-Sleep -Seconds 2

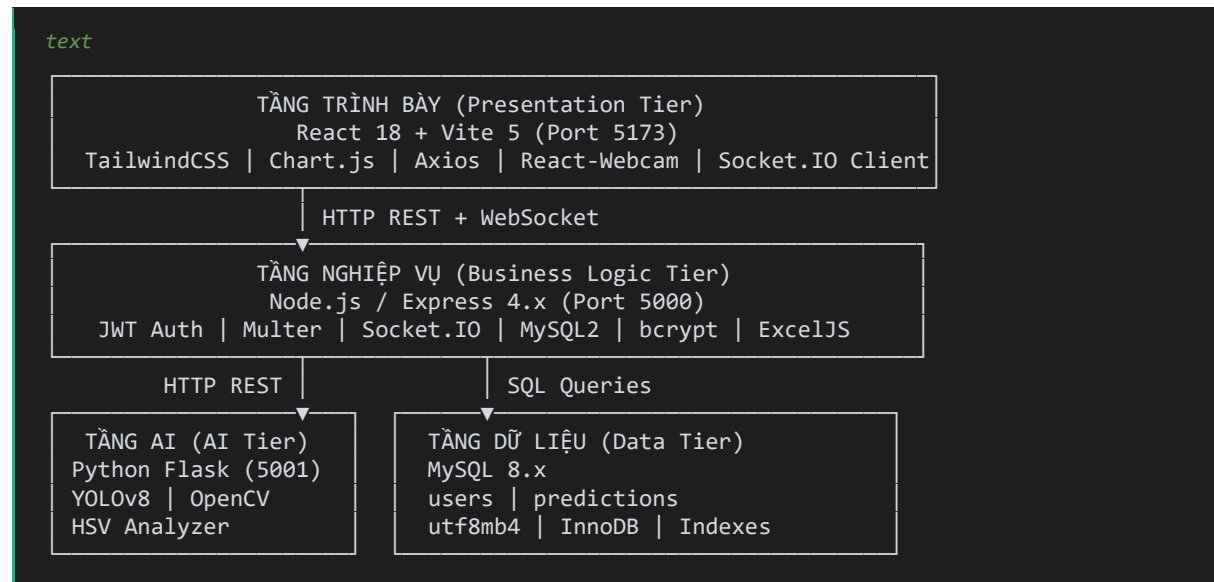
# Khởi động Frontend React/Vite
Start-Process powershell -ArgumentList `
    "-NoExit -Command `\"cd d:\aicode\frontend; npm run dev`\"" -WindowStyle
Normal

Write-Host "All services started!" -ForegroundColor Green
Write-Host "Frontend: http://localhost:5173" -ForegroundColor Yellow
Write-Host "Backend: http://localhost:5000" -ForegroundColor Yellow
Write-Host "AI API: http://localhost:5001" -ForegroundColor Yellow
```

3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

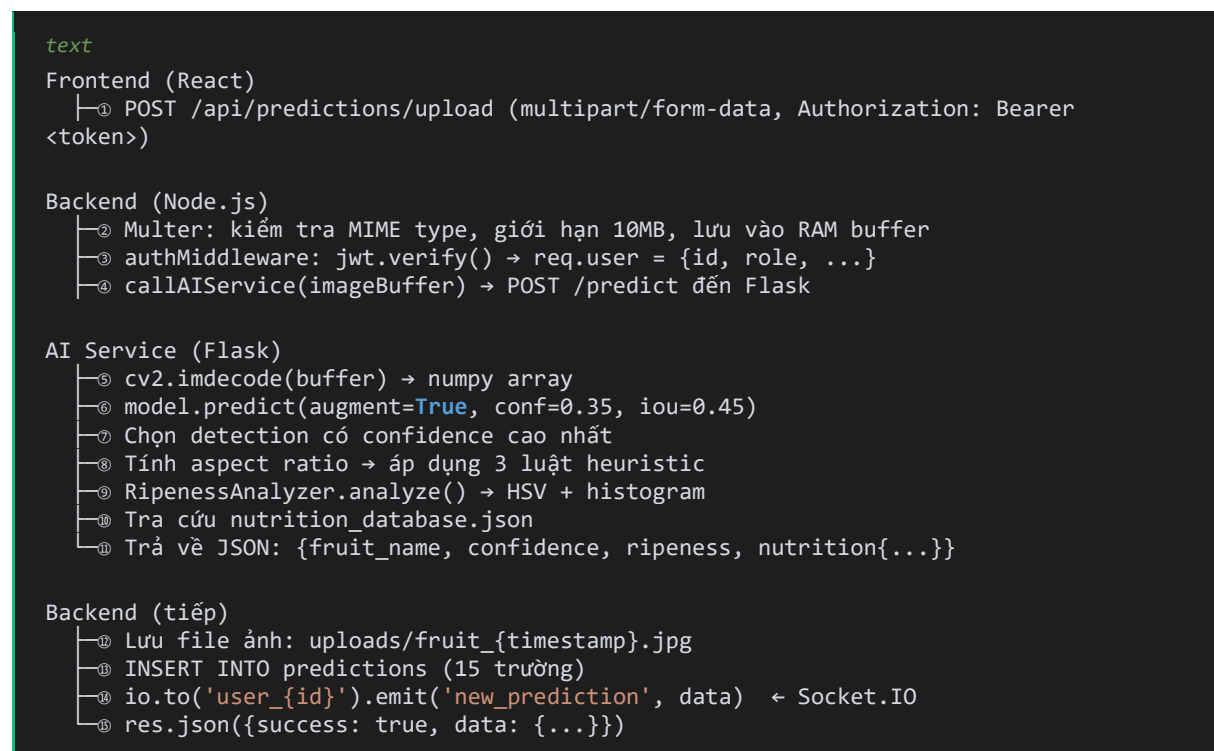
3.1. Kiến trúc tổng thể (Microservice 3-tier)

Hệ thống được thiết kế theo mô hình **Microservice 3 tầng** với sự phân tách hoàn toàn về trách nhiệm và công nghệ:



3.2. Luồng dữ liệu chính

3.2.1. Luồng nhận diện ảnh upload



3.3. Phân tích yêu cầu hệ thống

3.3.1. Yêu cầu chức năng

Mã	Chức năng	Mô tả
UC01	Đăng ký	Nhập tên, email, mật khẩu ≥ 6 ký tự; email không trùng
UC02	Đăng nhập	Email + mật khẩu $\rightarrow$ JWT token 7 ngày
UC03	Cập nhật hồ sơ	Đổi tên hiển thị và/hoặc mật khẩu
UC04	Upload ảnh	JPG/PNG/WebP/BMP, tối đa 10MB
UC05	Nhận diện webcam	Chụp ảnh realtime từ camera thiết bị
UC06	Xem kết quả	Tên quả, % confidence, độ chín, 15 chỉ số dinh dưỡng
UC07	Xem lịch sử	Phân trang 10/trang, tìm kiếm tên quả, lọc theo ngày
UC08	Xóa bản ghi	Xóa 1 bản ghi hoặc xóa tất cả (kèm file ảnh)
UC09	Xuất Excel	File .xlsx với 9 cột, header màu xanh lá
UC10	Xuất PDF	File PDF landscape với autoTable
UC11	Dashboard	Biểu đồ 7 ngày, 12 tháng, phân phối loại quả
UC12	Realtime	Socket.IO cập nhật lịch sử ngay sau nhận diện
UC13	Admin - User	Xem danh sách user + số lần nhận diện, xóa user
UC14	Admin - Data	Xem tất cả bản ghi hệ thống, xóa theo ID
UC15	Admin - Stats	Thống kê: tổng user, tổng nhận diện, hoạt động gần đây

3.3.2. Yêu cầu phi chức năng

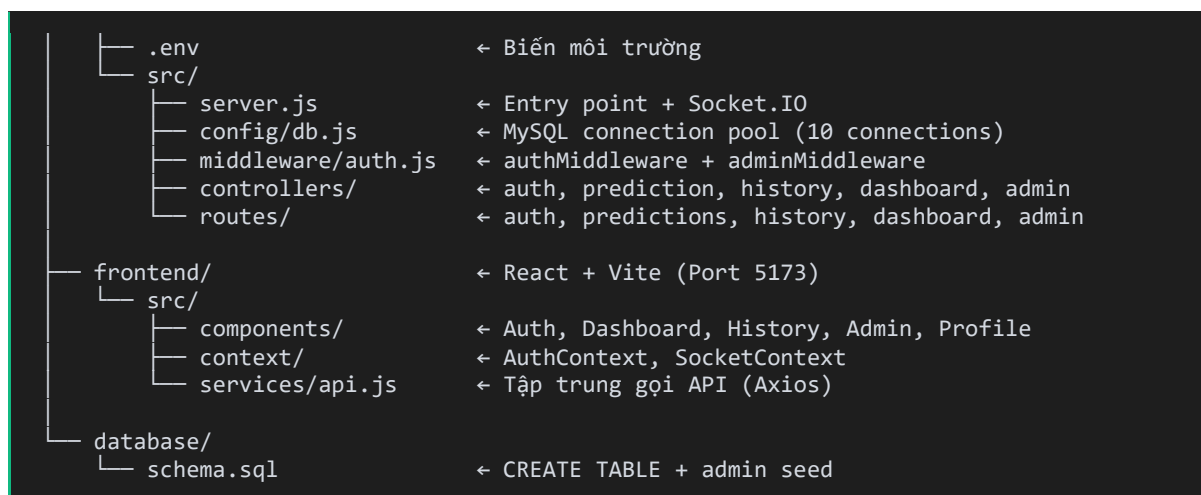
Tiêu chí	Yêu cầu	Cách đạt được
Bảo mật	Mật khẩu không lưu plaintext	bcrypt hash cost factor 10
Bảo mật	API endpoints được bảo vệ	JWT middleware + role check
Hiệu năng	Inference < 3s trên localhost	YOLOv8n/s, augment=True
Khả năng mở rộng	Thêm loại quả mới không sửa backend	Cấu trúc JSON nutrition DB
Độ tin cậy	Không trả kết quả sai confidence thấp	Threshold 35%
Tính toàn vẹn dữ liệu	Xóa user xóa luôn toàn bộ dữ liệu	ON DELETE CASCADE
Xử lý lỗi	Lỗi AI không crash backend	try/catch + 422 status code
Dung lượng upload	Giới hạn 10MB	Multer fileSize limit

3.4. Cấu trúc thư mục dự án

```

text
d:\aicode\
├── best.pt                ← Model YOLOv8 fine-tuned (6.3 MB)
├── nutrition_database.json ← CSDL dinh dưỡng 30 loại quả (USDA)
├── start_all.ps1          ← Script khởi động 3 dịch vụ
├── ai_service/
│   ├── app.py             ← Python Flask AI Service (Port 5001)
│   ├── model_service.py   ← Flask server: /health + /predict
│   └── requirements.txt    ← YOLOv8 + RipenessAnalyzer + heuristic
└── backend/               ← Node.js Express (Port 5000)

```



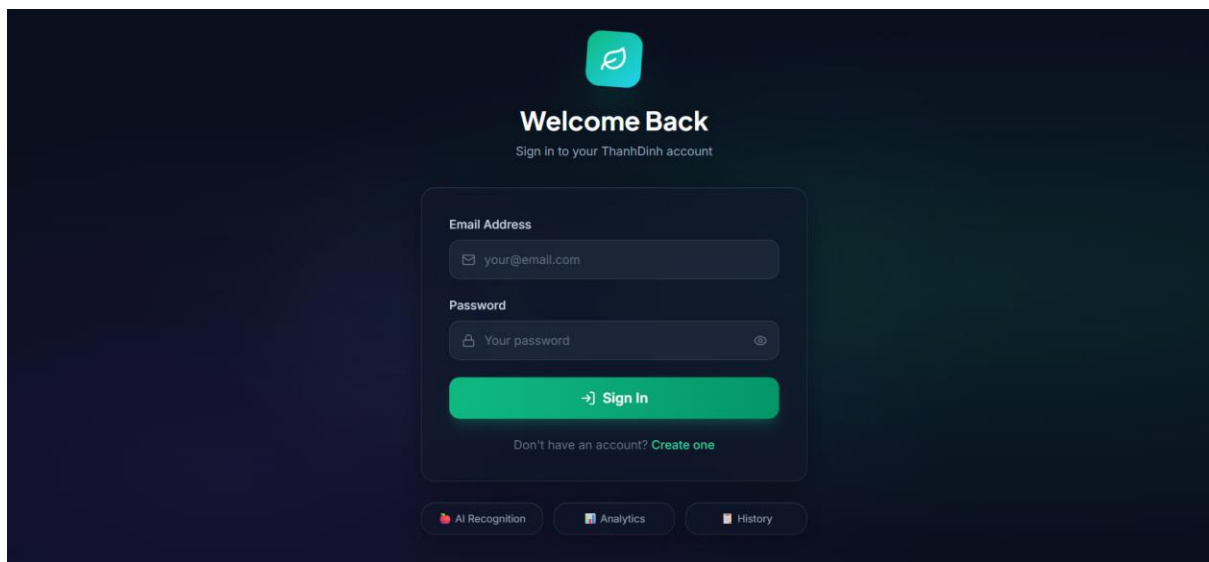
4. KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM

4.1. Môi trường triển khai

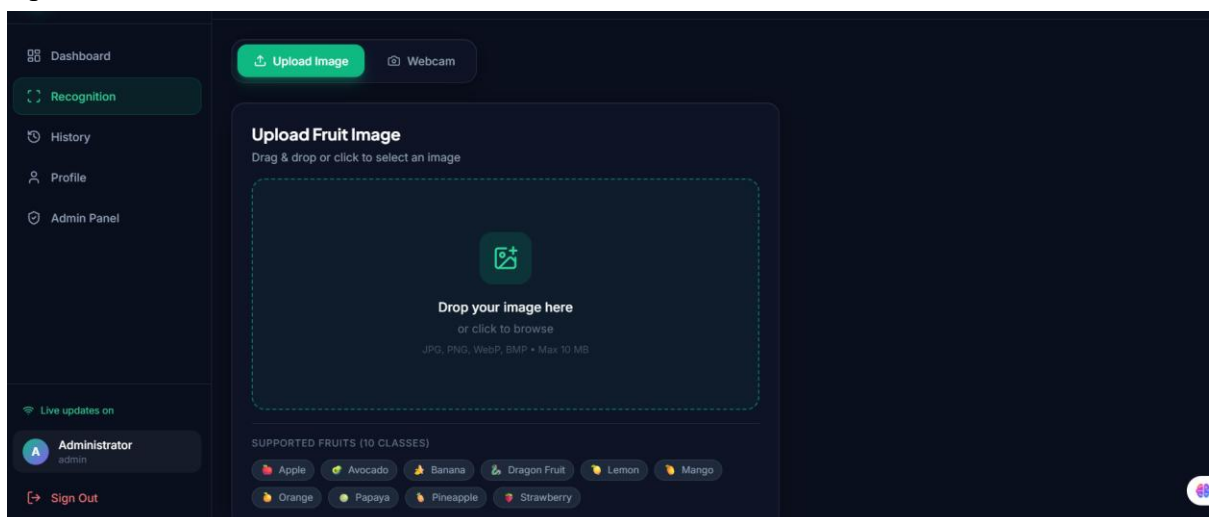
Thành phần	Phiên bản	Ghi chú
Python	3.10	Môi trường Anaconda tdenv
PyTorch	≥ 2.0.0	CPU inference mode
Ultralytics	8.2.79	YOLOv8 framework
OpenCV	4.10.0.84	Xử lý ảnh và phân tích màu
Flask	3.0.3	AI Service web server
Flask-CORS	4.0.1	Cross-origin resource sharing
Node.js	20.x LTS	Backend runtime
Express	4.x	Web framework
Socket.IO	4.x	Realtime WebSocket
mysql2	3.x	MySQL driver với Promise API
bcryptjs	—	Mã hóa mật khẩu
jsonwebtoken	—	JWT authentication
ExcelJS	—	Xuất file Excel .xlsx
jsPDF	—	Xuất file PDF
MySQL	8.x	Cơ sở dữ liệu quan hệ
React	18.x	Frontend framework
Vite	5.x	Build tool và dev server
TailwindCSS	3.x	Utility-first CSS
Chart.js	—	Biểu đồ thống kê
OS	Windows 10/11	Môi trường phát triển

4.2. Kết quả giao diện hệ thống

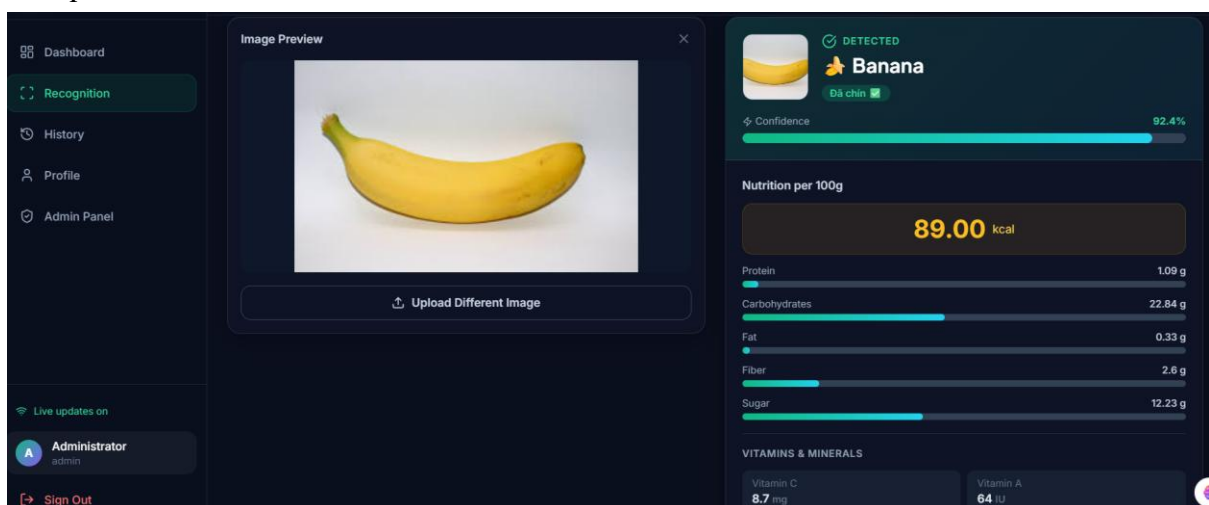
Đăng nhập:



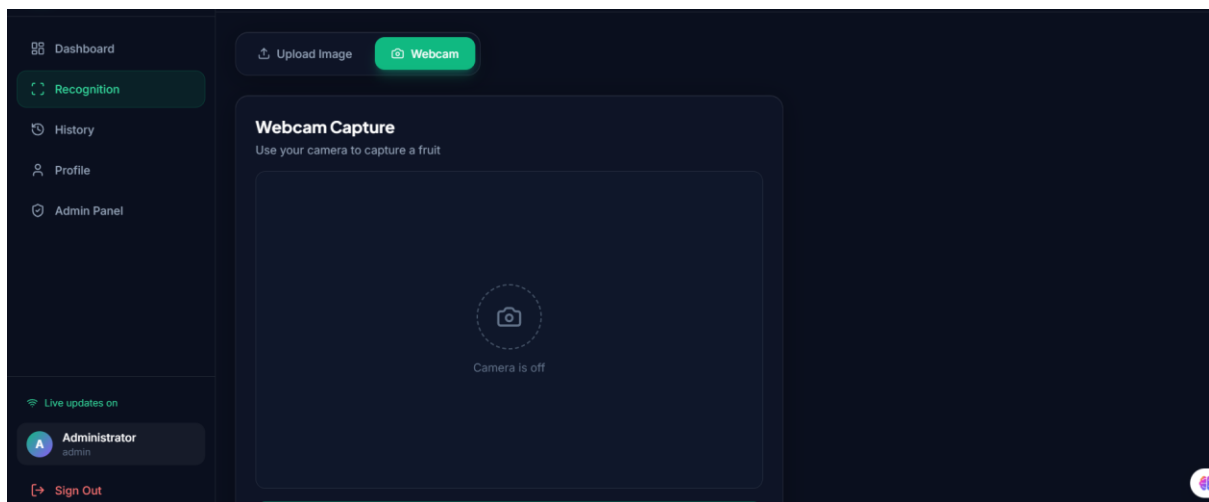
Upload ảnh:



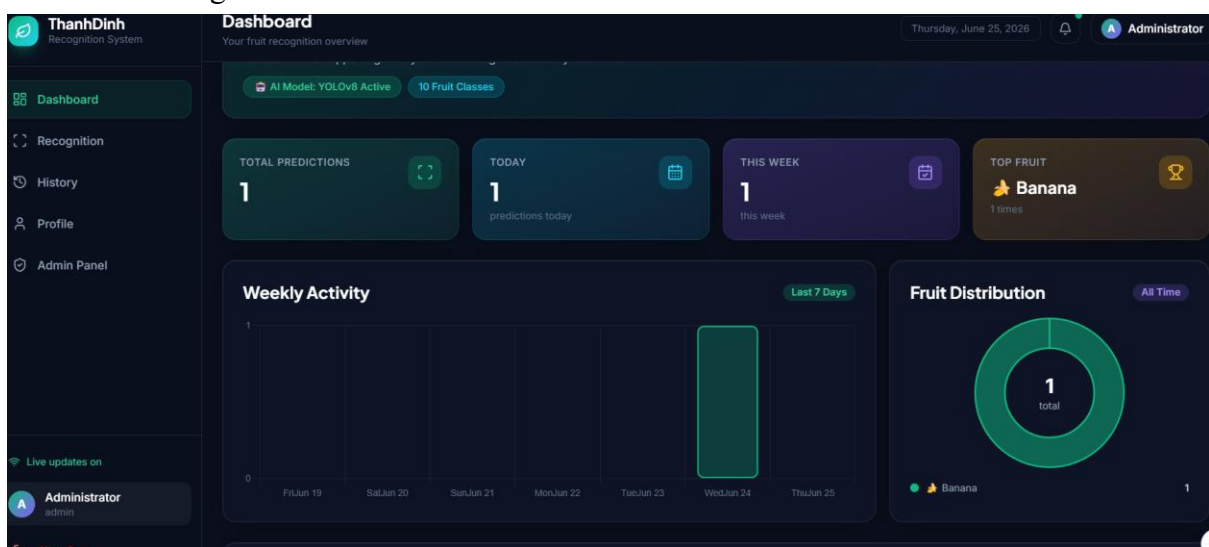
Kết quả nhận diện



Webcam realtime:



Dashboard thông kê:

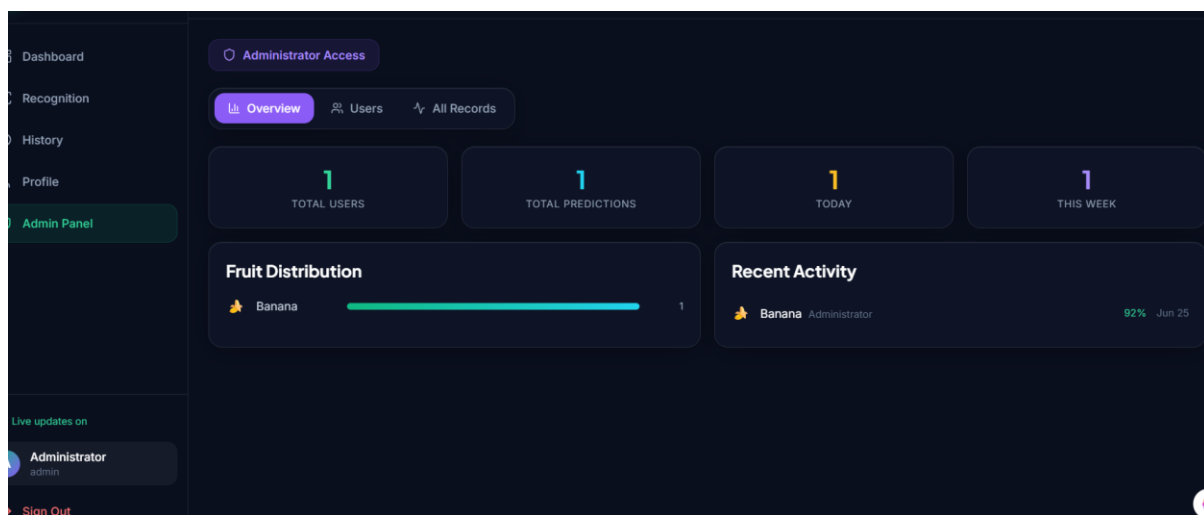


Lịch sử :

The screenshot shows the 'History' section of the Recognition System. It features a search bar with a placeholder 'Search fruit name...' and date filters. Below the search bar, it indicates '1 records found'. The table below lists the recognition results for a Banana.

IMAGE	FRUIT	CONFIDENCE	RIPENESS	CALORIES	PROTEIN	DATE	ACTIONS
	Banana	92.4%	Đã chín	89.00 kcal	1.09g	Jun 25, 2026 17:43	

Admin pannel:



4.3. Bảng kết quả thử nghiệm nhận diện

STT	Loại quả thực tế	Trạng thái thực tế	Kết quả AI	Độ chín AI	Confidence (%)	Đúng/Sai
1	Chuối	Chín	Chuối	Chín	92,4	Đúng
2	Táo	Chín	Táo	Chín	95,4	Đúng
3	Cam	Chín	Cam	Vừa	81,3	Đúng
4	Thanh long	Vừa	Thanh long	Vừa	80,8	Đúng
5	Xoài	Vừa	Xoài	Vừa	82,5	Đúng
6	Dây tây	Chín	Dâu tây	Chín	45,5	Đúng
7	Bơ	Chưa chín	Bơ	Chưa chín	96,3	Đúng
8	Dứa	Chín	Dứa	Quá chín	89,7	Đúng
9	Chanh	Vừa	Táo	Chưa chín	84,2	Sai
10	Đu đủ	Chín	Xoài	Chín	94	Sai

4.4. Các trường hợp biên đã xử lý

Trường hợp	Hành vi hệ thống
Upload file không phải ảnh (.pdf, .txt)	400 Bad Request + thông báo lỗi MIME type
Upload ảnh > 10MB	413 Request Too Large
Ảnh không chứa trái cây	422 + {"success": false, "message": "No fruit detected"}
JWT hết hạn	401 Unauthorized
User không phải admin truy cập admin API	403 Forbidden
Model best.pt không tìm thấy	Flask raise FileNotFoundError, AI Service không khởi động
MySQL mất kết nối	process.exit(1) — Backend thoát và thông báo lỗi

5. KẾT LUẬN

5.1. Những kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và cài đặt, đồ án **FruitAI** đã hoàn thành các mục tiêu đề ra:

Về mặt kỹ thuật:

- **AI Service hoạt động ổn định:** Model YOLOv8 fine-tuned thành công nhận diện 10 loại trái cây với ngưỡng confidence 35%, kết hợp Test-Time Augmentation tăng độ chính xác.
- **Phân tích độ chín:** Thuật toán HSV Color Histogram với bộ luật heuristic tinh chỉnh kết quả dựa trên hình dạng, giảm thiểu nhầm lẫn giữa các loại quả tương tự.
- **Hệ thống full-stack đồng bộ:** Ba tầng dịch vụ (Python Flask + Node.js + React) hoạt động liên thông qua REST API và Socket.IO realtime.
- **Bảo mật đầy đủ:** Xác thực JWT, mã hóa mật khẩu bcrypt, phân quyền admin/user, giới hạn file upload.
- **Xuất báo cáo:** Hỗ trợ xuất cả Excel (.xlsx có styling) và PDF (landscape với autoTable).

Về mặt ứng dụng:

- Cung cấp thông tin dinh dưỡng chuẩn USDA cho 10 loại quả với 15 chỉ số.
- Hỗ trợ đặc biệt người dùng có nhu cầu y tế: tiểu đường, dị ứng, suy thận, thể hình.
- Lưu trữ lịch sử nhận diện với đầy đủ chức năng tìm kiếm, lọc, phân trang.

5.2. Hạn chế còn tồn tại

5.2.1. Vấn đề nhận diện ảnh chứa nhiều quả

Đây là hạn chế quan trọng nhất. Hàm `predict_image()` chỉ lấy detection có confidence cao nhất (`detections[0]` sau khi sort giảm dần). Toàn bộ các quả còn lại bị bỏ qua. Ngoài ra, ROI được cắt từ bounding box của quả được chọn, nhưng các quả xung quanh có thể lọt vào vùng ROI, làm màu sắc phân tích bị lẫn lộn.

5.2.2. Phân tích độ chín chịu ảnh hưởng từ ánh sáng

Thuật toán HSV hoàn toàn phụ thuộc vào màu sắc thực sự của ảnh. Ánh đèn vàng/đỏ khiến quả xanh bị nhận nhầm là đã chín. Ngược sáng hoặc ánh sáng huỳnh quang làm giảm số pixel hợp lệ → kết quả "Không rõ".

5.2.3. Các hạn chế khác

- **Chưa có tính toán dinh dưỡng theo khối lượng thực tế:** Toàn bộ dữ liệu trả về theo mức chuẩn 100g của USDA.
- **Số lớp nhận diện còn ít:** Chỉ 10 loại trái cây được hỗ trợ, trong khi `nutrition_database.json` đã có 30 loại.
- **Chưa có cảnh báo cá nhân hóa:** Người dùng chưa thể khai báo tình trạng sức khỏe để nhận cảnh báo phù hợp.

5.3. Hướng phát triển trong tương lai

Tính năng	Mô tả
Multi-detection	Trả về kết quả cho TẤT CẢ quả trong ảnh
Tính toán theo khối lượng	Nhập gram → tính dinh dưỡng chính xác
Cảnh báo sức khỏe cá nhân hóa	User khai báo bệnh nền → tự động cảnh báo
Mở rộng tập nhận diện	Thêm 15-20 loại quả Việt Nam

Phân tích độ chín CNN	Dùng classification CNN thay HSV thuần túy
Ứng dụng di động	React Native / Flutter với camera thiết bị
Instance Segmentation	YOLOv8-seg để tách rõ từng quả

5.4. Kết luận chung

Hệ thống AI nhận dạng và phân tích chỉ số dinh dưỡng của các loại trái cây đã được xây dựng thành công với kiến trúc microservice rõ ràng, tích hợp đầy đủ các công nghệ AI và web hiện đại. Hệ thống đáp ứng tốt các yêu cầu chức năng ban đầu, đặc biệt với người dùng có nhu cầu đặc biệt về sức khỏe dinh dưỡng.

Mặc dù còn hạn chế trong việc xử lý ảnh chứa nhiều quả, đây là bài toán kỹ thuật có hướng giải quyết rõ ràng trong tương lai (multi-detection, instance segmentation). Nền tảng kiến trúc hiện tại đủ vững chắc để tiếp tục phát triển thành một sản phẩm hoàn chỉnh, đóng góp thiết thực vào mục tiêu chăm sóc sức khỏe cộng đồng trong kỷ nguyên số.

TÀI LIỆU TRÍCH DẪN

- [1] G. Jocher, A. Chaurasia và J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] S. J. Pan và Q. Yang, "A Survey on Transfer Learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.
- [3] M. Al-Sarayreh et al., "Determining the ripeness of fruits using deep learning and color analysis," Journal of Food Engineering, vol. 264, 2020.
- [4] U.S. Department of Agriculture (USDA), "FoodData Central API & Database," 2023. [Online]. Available: <https://fdc.nal.usda.gov/>
- [5] OpenCV Documentation, "Color Space Conversions — HSV," 2024. [Online]. Available: <https://docs.opencv.org/4.x/>
- [6] M. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [7] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems 32, 2019.
- [8] Flask Documentation, "Flask Web Framework v3.0," Pallets Projects, 2024.
- [9] Socket.IO Documentation, "Real-time bidirectional event-based communication," 2024. [Online]. Available: <https://socket.io/docs/v4/>
- [10] JSON Web Token Specification, "RFC 7519 — JSON Web Token (JWT)," IETF, 2015.
- [11] bcryptjs, "A library to help you hash passwords," npm. [Online]. Available: <https://www.npmjs.com/package/bcryptjs>
- [12] ExcelJS Documentation, "Read, manipulate and write spreadsheet data," 2024. [Online]. Available: <https://github.com/exceljs/exceljs>
- [13] jsPDF Documentation, "Client-side JavaScript PDF generation," 2024. [Online]. Available: <https://github.com/parallax/jsPDF>
- [14] Roboflow, "Computer Vision Dataset Management, Annotation and Model Training," 2023. [Online]. Available: <https://roboflow.com/>